

Creating with Claude

A Practical Guide for Everyone

Chris Thatcher

EARLY READER PREVIEW — NOT FOR DISTRIBUTION

This is an uncorrected advance copy for review purposes only.

Content may differ from the final published edition.

Please do not quote, share, or distribute.

Creating with Claude: A Practical Guide for Everyone

Copyright © 2026 Chris Thatcher. All rights reserved.

Published by Marque Publishing

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

Claude is a product of Anthropic, PBC. This book is an independent work and is not affiliated with, sponsored by, or endorsed by Anthropic. All product names, trademarks, and registered trademarks are the property of their respective owners.

The information in this book is provided for educational purposes only. The author and publisher make no representations or warranties with respect to the accuracy or completeness of the contents.

First Edition: July 2026

marquepublishing.com

PREVIEW COPY — NOT FOR DISTRIBUTION

Contents

Introduction

Chapter 1: Meet Claude — and make it yours

Chapter 2: Your first real project

Chapter 3: Trust but verify

Chapter 4: Go deep

Chapter 5: See it

Chapter 6: Present it

Chapter 7: Claude on your computer

Chapter 8: Connect your world

Chapter 9: Build something that lives on its own

Chapter 10: Make it yours forever

Appendix: Your Quick Reference Guide

Introduction

I was a skeptic. My son was downright anti-AI. I have been working with technology most of my adult life. He grew up on it and was an accomplished programmer. I am not.

I'd been playing with ChatGPT for about a year, but "playing" is the right word. I treated it like Google on steroids — quick research, how-to questions, the occasional summary of something I didn't feel like reading. I was basically paying \$20 a month for a super search engine. It never occurred to me that AI could actually build things.

Then I met Chuck Poole at my local bar poker league.

Chuck is a security guy like me. He'd sold his last company and was figuring out what he wanted to do next. What he settled on was going all-in on AI. Pun intended. By the time I went to visit him, he'd been fully immersed for months. He was using AI to automate everything in his life. He used it to organize his thinking and draft new ideas for his blog. He even built a digital personal assistant he named Jarvis that wired all of his productivity tools together to simplify his increasingly complex life. But the part that got my attention was this: he was building things. Real things. Applications and websites for his friends. Tools that actually worked and were commercially viable.

I was blown away at what he described to me over the course of a few conversations, so I asked if he'd show me.

The following Saturday morning, we met for breakfast, and then I followed him home to see where the magic happened. He told me to bring my laptop because we were getting me set up to run Claude myself. I was skeptical, but I did as instructed.

I didn't follow a lot of what we did that morning, because it all happened so fast. Chuck got Claude installed on my machine along with some basic plugins and tools and got me started. Four hours flew by before I knew it, and I left his house intrigued by what I'd just opened up for myself.

Later that week, we talked on the phone and I told him what I'd been working on. I was excited. Then we started talking about poker — something we both love. I asked him if he thought Claude could help us build a poker training app.

He said, "I don't know. Let's find out."

That night, an obsession was born.

Over the next forty-five minutes or so, we laid out the parameters of what we wanted to build — for whom, and why. Then there was a two-hour session with Claude, creating the first prompt and refining the outputs until Claude was clear on the vision. Chuck already had all of the back-end

systems wired up, so he told Claude to build it and went to bed.

He woke up to a working MVP — a minimum viable product. I was not expecting what he shared with me in the morning. The app worked. It didn't do a lot, but it actually worked.

Over the next few weeks, we kept sharing ideas, and both of us were working on the code. It was the most fun I'd had in years. We talked about the commercial viability of the app, all the cool things we could do with it. We got it to a point where we were sharing it with friends, and they were actually using it.

Then I got an idea.

I'd been carrying around a couple of book ideas for years. I had notes and outlines and lots of emails to myself and lots of rants in Word documents about my vision — but no real progress in more than fifteen years on either one. In December, a mentor and good friend gave me a nudge and told me it was time to write my book.

Then the question that literally changed my life. Could Claude help me organize my thoughts and build the framework for a book?

To be clear — Claude didn't write my books. It was my guide through the process. It took a lot of trial and error to get the rules down, but I was able to go from a pile of notes and a thesis to a finished manuscript in about three months — most of that learning how to work with Claude. That is to say, three months after getting started with Claude, I had a book ready to publish, a novel in progress, and the poker project had branched into two more apps and a third book.

I am doing more at sixty-two than I ever have before.

If I can do it, so can you. Start thinking big and dusting off your dreams. Claude can help you bring them to life.

That's what this book is about. I'm going to walk you through everything I wish someone had walked me through — from your first conversation with Claude to building things you didn't think were possible. No code required. No technical background. Just you, a laptop, and a willingness to try. I can't teach you everything — most of it you will figure out on your own. But if you follow along and put in the time, you will find the sky is your limit and Claude is your new best friend.

Please make sure your seat backs and tray tables are in their upright and locked positions, because we are getting ready to take off.

Chapter 1: Meet Claude — and make it yours

The Story

The first time I sat down with Claude on my own, without Chuck looking over my shoulder, I didn't have a plan. So, I stared at the prompt for a few minutes wondering where to start. I had a head full of ideas and no idea what to do with them.

I'd been carrying around project ideas for years. Book concepts, software ideas, and business ideas I kept sketching on napkins. They lived in random notes and emails to myself and late-night Word documents that never went anywhere. So, I did what Chuck encouraged me to do when I didn't know where to start. I just started dumping them into Claude.

Not as a list, as a conversation. I told Claude about my background in security. I told it what I am good at and what I am not good at. I shared the books I'd been wanting to write, the software projects and game ideas I thought had legs. I gave it context about my experience, my goals, and my perspective. As I kept talking, something started happening.

The more I explained, the more useful the responses got. The first thing that stood out to me through the process was, I have a lot to say. The second thing that landed is, the more context I gave Claude, the better the interaction became. I went from dropping random thoughts into a prompt with abstract reasoning, to well-thought-out concepts that had possibilities. My book ideas went from outlines and a bunch of disjointed notes I had made over the years to clear visions I could execute on. Similarly, with the poker app, I was creating by talking to Claude the way I would talk to a contractor I hired to do a job.

Claude pushed back on some ideas and asked questions about others. With each round of iteration, my thoughts became clearer and things that had been ideas just a few hours ago were taking shape into things I could actually bring to life. There was one conversation in particular that connected two projects I'd been thinking about separately in a way I hadn't considered. By the end of that session, I had ten viable project ideas organized by feasibility and effort. That's ten things I could actually start. That was the moment I realized Claude could connect the dots between me and my abstract ideas and be the vehicle to engage in a way that could make them real.

I'd walked in with a pile of scattered thoughts. I walked out with a plan; ten of them actually. Well, two were rabbit holes most people who start using AI go down. Figuring out how to make money in the stock market and the bigger, how do I generate passive income streams. What made them different from the ones that got legs and are actively being worked on or tabled for the future, is they were just about making money. They weren't me. My ideas and my visions were what drove meaningful interaction about what was already there. Claude just helped me reframe it, put scope around it, and

bring it to life.

It may sound crazy, but that first night after a marathon session at the keyboard, I felt an awakening of something I thought was gone. My creative side was fully engaged and I started dreaming about what is possible. 62 years old and getting a second chance to follow my dreams.

Build It: Installing Claude and Your First Conversation

By the end of this section, you'll have Claude installed on your computer and you'll have had your first real conversation with it. This isn't a read about what Claude can do kind of book. This is a roll up your sleeves and do stuff book. So, you are going to follow along and before the chapter is done, you will have built something that actually helps you do something productive.

Step 1: Download Claude

Open your web browser and go to **claude.ai**. You'll see a clean page with an option to sign up or log in.

Click **Sign up**. You can use your email address or sign in with Google. Either works, so pick whichever is easier for you.

Claude has a free tier that lets you have real conversations and do real work. There's also a Pro plan for \$20 a month that gives you more usage and access to the most powerful models. My advice: start free. You can do everything in this chapter without paying a dime. If you find yourself hitting limits and wanting more, upgrade then.

Behind the scenes, Claude has different models — Haiku, Sonnet, and Opus. Think of them as speed versus depth. Haiku is fast and light, Sonnet handles most everyday work, and Opus is the heavy hitter for complex reasoning and nuance. The free plan uses Sonnet. We'll get into when to use which one in Chapter 10, but for now just know they exist.

I will tell you that I signed up for the Pro Plan the morning I was at Chuck's and before the end of the session, I had maxed out my usage and upgraded to the Max Plan. Your personal mileage may vary.

Once you have an account, we're going to install the desktop app. The desktop app is how I use Claude every day, and it's how you'll get the most out of this book. The desktop app can see your files, work with your documents, and eventually connect to the tools you already use. The web version is fine for quick questions, but the desktop app is where you'll want to be as we get deeper into the book.

Go to **claude.ai/download** and download the app for your computer — Windows or Mac, it doesn't matter. Install it the way you'd install any other application. Open it up and sign in with the account you just created.

You should be looking at a clean conversation window with a text box at the bottom.

My first reaction was okay, now what. There was a lot of stuff on the screen that didn't make sense yet. Don't focus on any of that. It helps to start with an idea of what you want the outcome to look like, but this is your first time. We are going to jump right in.

You shouldn't, but if by chance you have problems, don't despair. You can sign in to claude.ai in your browser and use the snip tool to capture pictures of error messages or anything that is confusing and copy them into the chat prompt. Just say, I am new to all this and ask what do I do now? It will tell you exactly what to do and patiently answer all of your questions. Once you are up and running, you will find yourself asking Claude to explain things you don't understand a lot.

Step 2: Say Something Real

Here's where most people go wrong with AI. They type "Hello" or "What can you do?" or "Tell me a joke." Claude will answer all of those, politely. But you didn't install this thing to make small talk.

Think of something you've been meaning to get done. It should be meaningful to you. It doesn't have to be complicated — in fact, simpler is better for your first conversation. Here are a few ideas to get your wheels turning:

- An email you've been putting off writing
- A summary of something you need to explain to someone
- A plan for a project you've been thinking about
- Advice on how to handle a tricky situation at work
- A list of questions you should be asking about something important
- A group of documents or data you need to synthesize but have been putting off because of how long it would take

Now type it into that text box the way you'd explain it to a smart friend who's never heard of your problem before. Don't worry about formatting and don't worry about grammar or punctuation. Just talk to it. And yes, you can actually talk to it. I prefer typing, but the microphone is there if you prefer to dictate.

Here's an example. Let's say you've been putting off writing a difficult email to a client about a project delay. You might type something like this:

I need to write an email to my biggest client. We're three weeks behind on their project because we lost a key team member, and I need to let them know without making it sound like we're falling apart. The client is Sarah Chen, VP of Operations at Customer Group. She's direct and doesn't like surprises. Can you help me draft something?

Hit Enter and watch what happens.

Claude is going to come back with a draft that's surprisingly good. It'll probably ask you a few clarifying questions too — things like the tone you want, whether you have a new timeline, whether you've already discussed this verbally. That's normal. That's Claude making sure it understands what you actually need before it delivers. These are also the kind of details that you will learn quickly to

include in your prompt.

Answer the questions. Read the draft. This is not the final output; it is the beginning of the conversation. Tell Claude what you like and what you'd change.

This is the part that surprises people: you can just *talk back to it*. Say "Make it shorter" or "That sounds too formal" or "Actually, I forgot to mention that we've already brought in a replacement." Claude adjusts. Each exchange gets you closer to the outcome you want.

I should point out this is also where most people get it wrong and decide AI just isn't there yet. They create a very detailed prompt and expect a perfect response the first time. AI doesn't work like that. The first response is the starting point. Each iteration shapes the output into something better until you have something you can use.

I can remember more than once thinking I thought this thing was supposed to be smart. Instead, it literally made something up. When I asked if it fabricated anything in the last response, it said yes! So, it isn't perfect and you will need to verify anything you need to rely on.

Go back and forth until you have something you'd actually send. That might take two exchanges or it might take five. Either way, you just did something productive with Claude in your first sitting.

Step 3: Try One More Thing

Think of something completely different from what you just did. If you wrote an email, try asking Claude to help you brainstorm. If you brainstormed, ask it to help you organize something. The point is to see that this thing can handle way more than you are thinking about right now. It's not just an email writer or a planner. It adapts to whatever you bring it.

Think of Claude as your second brain — your advisor and sounding board. The more you challenge it, the more it will challenge you and the outputs you end up with will reflect it.

Here's another example. Let's say you coach your kid's soccer team and you need to plan the next few practices:

I coach a U-12 soccer team. We have 14 kids, mixed skill levels. Our next game is Saturday and we have two practices before then — Tuesday and Thursday, 90 minutes each. The team's biggest weakness is passing under pressure. Can you help me plan both practices?

Just talk to it like a person. Give it context. Tell it what matters.

Spend fifteen or twenty minutes with this second conversation. By the time you're done, you'll have two real, useful outputs from Claude, and you'll already have a feel for how to work with it.

One more thing — Claude will occasionally get something wrong. You will get a suggestion that doesn't fit your situation or a tone that totally misses the mark. That's normal and it's not a reason to panic. You'll learn to spot it, and we'll spend all of Chapter 3 on how to verify what Claude gives you. For now, just know that pushing back works. If something's wrong, say so. Claude corrects.

There are times when AI doesn't have a good answer, so it just makes things up. You will get in the habit of asking if it made up something that seems off. Don't be surprised if it says yes. As we go deeper into the book, we will talk about building custom rules to reduce the frequency and catch it when it gets it wrong.

What Just Happened

Here's what I noticed in my own first sessions that I didn't expect.

I figured interacting with Claude would be like using a search engine. In other words, ask a question, get an answer. What I didn't expect was that the *conversation* was where the magic happens. The deeper the conversation, the better the outcome. Every time I pushed back or added context, the output got sharper.

When you told Claude about Sarah Chen being direct and not liking surprises, Claude used that to shape the tone of the email. When you told it about fourteen kids with mixed skill levels, Claude used that to plan age-appropriate drills. You probably didn't think of those details as "prompting techniques." You were just being specific about your situation. That's the whole trick.

I remember an early session that wasn't giving me the outcome I was looking for. I was seriously wondering if Artificial Intelligence was an oxymoron. I kept trying to refine a message and after the fourth or fifth iteration it was still not what I was looking for. After reviewing the entire chat, it occurred to me that I was not really clear on what my end in mind was. We spent a couple hours shooting at a target we hadn't defined properly. A couple minor additions to the conversation and I got exactly what I was looking for. In hindsight, I could have just asked Claude what was missing from the conversation and why I wasn't getting what I wanted out of the chat. That's what I do now.

In my experience, people overcomplicate this. They read articles about prompt engineering and think they need to learn a special language. You don't. You just need to tell Claude what you'd tell a colleague if you were handing off the work. Tell it who it's for, what matters, what to avoid. Claude will do the rest.

Success with AI does not necessarily belong to the smartest or most technical people at the keyboard. The people who make AI work for them are the ones that can clearly articulate the problem and what a positive outcome looks like. You set the target and keep steering AI through the prompt until you get something useful. Then you make it better.

The other thing worth knowing: your first message doesn't have to be perfect. Mine rarely are. I start loose, see what comes back, and then steer. That back-and-forth is the process, not a sign that you did something wrong.

Now Go Make Something

You've got Claude installed and you've had your first real conversations. Now put the book down and go use it.

The best first project is the one that's been nagging at you. The thing on your to-do list that keeps getting pushed to tomorrow. Open Claude and start talking about it. Give it the context — who, what, why — and see what comes back. Push on whatever isn't right. Keep going until you have something you'd actually use.

Do that before you turn to Chapter 2. The people who get the most out of this book are the ones who put it down between chapters and go build something.

— — —

Quick Reference Card

Getting started with Claude — the essentials

What	How
Sign up	Go to claude.ai and create a free account
Install the desktop app	claude.ai/download — Windows or Mac
Start a conversation	Type in the text box at the bottom and hit Enter
Give good context	Tell Claude who, what, why, and any constraints — like you'd brief a colleague
Iterate	Tell Claude what to fix: "shorter," "more formal," "I forgot to mention..."
Start a new conversation	Click the + icon for a fresh thread when you switch topics

Your first prompt template:

I need to [what you're trying to accomplish]. The context is [relevant background]. The audience is [who will see this]

Chapter 2: Your first real project

The Story

A few months ago, I had to present to the CEO and his executive team on the impact of AI on cybersecurity. With all the hype around Mythos, he wanted our point of view — something he could take to client executives and speak intelligently about.

Our team had spent weeks on it. Compiling research, debating conclusions, refining the message. We distilled everything down to a sixteen-page document. It was thorough. It was well-sourced. And it was completely wrong for the room.

We had a thirty-minute time slot, but anyone who's presented to a CEO knows the math. You have ten minutes. Maybe. Three slides, tops. The rest is discussion — if you're lucky enough to get that far.

In previous presentations to this CEO, we had never finished. He'd cut in, redirect, or run out of patience before we got to the point. So, the stakes weren't just the content. The stakes were whether he'd let us get through it at all.

I sat down with Claude and gave it the context. Here's our research — sixteen pages. Our audience is the executive leadership team of a five-billion-dollar engineering company. Synthesize this and give me the most important data for that audience. I told it I needed to answer three questions: What has changed? What is the risk and potential impact of AI on security? What should companies be doing now? Then I asked it to identify the opportunity for our company.

Claude distilled sixteen pages down to six. We talked through why it selected what it did, what to cut, how to frame the message so we could deliver it in ten minutes on three slides. It didn't nail the message the first time — the framing was too broad, too much data and not enough narrative. We went back and forth until the message was sharp enough to land in a room full of people who don't have time for preamble.

After that, it was building the slides and the script. We iterated on those too. It wasn't perfect, but it was close — and close was a starting point I could work with.

The presentation day came. The CEO listened. He nodded. He let us finish.

That had never happened before.

Then he did something I didn't expect. He gave our presentation back to us — in his own words. He said this is exactly what we need to be doing. Less than a week later, he told us he'd delivered the same talk to two different client CEOs. Both of them asked him to arrange meetings with their teams as soon as possible.

Sixteen pages of research became three slides and a ten-minute talk that booked two client meetings in a week.

That's what this chapter is about. Not how to ask Claude a question — you already know how to do that. This chapter is about taking something from nothing to done. From blank page to a finished product you'd actually put your name on.

Build It: Your First Real Project Plan

By the end of this section, you'll have a complete project plan — something you can actually follow, not just a wish list. You're going to build it from scratch with Claude, and by the time you're done, you'll understand the process of going from "I have an idea" to "I have a plan."

Pick Your Project

Before you open Claude, decide what you're building a plan for. Pick something real — something you actually want to do. This works best when the stakes are personal.

Here are two options. Pick whichever one speaks to you, or substitute something else entirely. The process is the same regardless.

Option A: A fitness and nutrition plan. Maybe you've been meaning to get in shape, lose weight, train for something, or just stop eating like a college freshman. You have a goal but no structure around it.

Option B: A plan to start a business. Maybe you've had an idea kicking around for months — a side hustle, a service, a product. You know what you want to do but haven't mapped out how to actually get there.

Pick one. If neither of those grabs you, pick anything that matters to you — planning a move, organizing a major event, launching a creative project. What matters is that you care about the outcome. If you don't care, the exercise is just typing.

Step 1: The Brain Dump

Open Claude and start talking. Don't organize your thoughts first. Don't try to write a clean prompt. Just unload everything in your head about this project.

If you picked the fitness plan, it might look something like this:

I want to get in better shape. I'm 45, about 30 pounds overweight, and I haven't worked out consistently in years. I have a gym membership I never use. My knees aren't great so running is probably out. I travel for work about one week a month. I eat out a lot — probably four or five times a week. My doctor told me my cholesterol is high and I should lose weight. I don't want a bodybuilding plan. I just want to feel better and not be out of breath walking up stairs. I have

about 45 minutes a day I could dedicate to this, mornings before work.

If you picked the business plan:

I want to start a small business doing home organization and decluttering. I've been doing it for friends for free and they keep telling me I should charge for it. I live in a mid-sized city, about 200,000 people. I'd want to start part-time since I still have my day job. I have about \$2,000 I could invest to get started. I have no idea how to price my services or find clients beyond word of mouth. My goal would be to replace my current income within two years so I could do this full-time.

Hit Enter.

Notice what you just did. You didn't ask Claude a question. You gave it everything bouncing around in your head — the goal, the constraints, the obstacles, the reality of your situation. That's not rambling. That's context, and context is the single most important thing you can give Claude.

Step 2: See What Comes Back

Claude is going to come back with something structured. It might be a preliminary plan, a set of questions, or both. Read the whole thing before you respond.

For the fitness example, Claude will probably suggest a phased approach — maybe starting with three days a week and building up. It might recommend low-impact exercises for the knees, suggest meal prep strategies for travel weeks, and flag that you should talk to your doctor before starting.

For the business example, you'll likely get an outline covering market research, pricing models, legal basics, marketing strategies, and a timeline.

Here's the important part: it won't be right. Not completely. Some of it will feel generic. Some of it will miss the mark on your specific situation. That's not a failure — that's the starting point.

Step 3: Push Back

This is where most people stop. They get the first response, think "that's pretty good," save it somewhere, and never look at it again. That's like writing the first draft of an important email and hitting send without reading it.

The real work starts now. Tell Claude what's wrong.

For the fitness plan, you might say:

The gym schedule won't work when I'm traveling. I need something I can do in a hotel room with no equipment. Also, I hate cooking and I'm not going to meal prep on Sundays — that's never going to happen. Give me options that work for someone who eats out most of the time.

For the business plan:

The pricing section feels generic. I looked at competitors in my area and they charge anywhere from \$50 to \$150 an hour. I don't know where I fit. Also, I'm not going to do social media marketing — I don't have the time or the patience. What else can I do to find clients?

This is the step that changes everything. You just told Claude what's real and what's not. You eliminated the parts that look good on paper but would never survive your actual life. Claude will rebuild around those constraints.

Go back and forth at least two or three more times. Each round, get more specific. Challenge the assumptions. Add details you forgot the first time. Ask why it made certain choices.

Step 4: Get Specific

Once the overall structure feels right, it's time to drill down. You need dates, numbers, and concrete actions — not general advice.

Try something like this:

This is getting closer. Now I need you to make it specific. Give me a week-by-week plan for the first 90 days. Include exactly what I should do each day — not categories, specific actions. For the nutrition part, give me actual guidelines I can follow when I'm sitting in a restaurant looking at a menu.

Or for the business:

I like the overall plan. Now break it down into a 90-day launch sequence. Week by week. What do I need to do first, second, third? Include specific tasks — not "develop a pricing strategy" but "call three competitors posing as a customer and find out what they charge." Give me things I can check off a list.

This is where Claude really earns its keep. It can take a broad plan and turn it into a day-by-day execution roadmap. The trick is telling it exactly how specific you need it to be. "Give me a plan" gets you a plan. "Give me a week-by-week breakdown with specific daily actions I can check off" gets you something you can actually follow.

Step 5: Stress Test It

Before you call this done, make Claude poke holes in it.

Look at this plan as a skeptic. What's most likely to fail? Where am I being unrealistic? What am I missing that someone who's actually done this would know?

This is one of the most underrated things Claude can do. It will tell you the hard truths — the parts of your plan that sound good but probably won't survive contact with reality. Maybe your timeline is too aggressive. Maybe you're underestimating costs. Maybe the workout plan assumes a level of discipline that doesn't match what you told Claude about your current habits.

Read the critique carefully. Some of it will sting because it's true. Adjust the plan.

Step 6: Polish and Finish

You've been going back and forth for a while now. The plan is specific, realistic, and stress-tested. Time to turn it into something finished.

Take everything we've built and give me a clean, final version. Organize it so I can print it out and follow it. Start with a one-paragraph summary of the goal, then the week-by-week plan, then a section of things to track or measure, then a list of things I need to buy or set up before I start.

Claude will produce a clean, organized document. Read through it one more time. If anything is off, fix it now. This is your last pass.

When you're happy with it, copy the whole thing into a Word document, a Google Doc, or wherever you keep things you actually plan to use. Don't leave it in Claude — put it somewhere you'll see it.

You just went from a vague idea to a finished plan you can execute. That's the process. It works the same whether you're building a fitness plan, a business plan, a book outline, a project proposal, or a presentation for a CEO who's never let you finish before.

What Just Happened

You just did something that most people think requires either an expert or a lot of time. You built a complete, specific, actionable plan from nothing — and you did it in one sitting.

Here's what I want you to notice about the process, because you'll use it for everything going forward.

You started by telling Claude your situation, not by asking a question. That brain dump in Step 1 wasn't wasted words. Every detail you gave — your age, your bad knees, your travel schedule, your \$2,000 budget — shaped everything that came after. The more context Claude has about you and your situation, the more useful the output. I've seen it over and over in my own work. The conversations where I front-load context are the ones where I get something I can actually use.

Then you pushed back. You told Claude what wouldn't work and why. Every time you said "that won't work because..." you eliminated a path Claude didn't know was a dead end. The plan got better because you made it more honest.

You asked for specifics instead of accepting generalities. "Give me a plan" is a different request than "give me a week-by-week breakdown with daily actions I can check off." Claude will give you whatever level of detail you ask for. If you don't ask for specifics, you'll get advice. If you do, you'll get a roadmap.

And you stress-tested it before calling it done. You asked Claude to be a skeptic and tell you where the plan would break. That's the step that separates a plan you'll follow from a plan you'll forget about by Thursday.

In my experience, this process — dump, structure, push back, get specific, stress test, polish — works for almost any project. The CEO presentation I opened this chapter with followed the same steps. Sixteen pages of research, dumped into Claude with context about the audience. Structure it for three slides and ten minutes. Push back on what didn't fit. Get specific about the message. Stress test whether it would land with this particular CEO. Polish the script. Done.

The blank page isn't the hard part. The hard part is knowing how to get from blank to done. You just did it.

Now Go Make Something

You've built one project plan. Now build another — for something completely different.

The point of this chapter isn't fitness plans or business plans. The point is the process. And the only way to make the process yours is to use it again. And again.

Here are ideas to get you started. Pick one that fits your world and go build it right now, before you move to Chapter 3.

If you run a business or manage people: Build a 90-day onboarding plan for a new hire. Build a client proposal for a prospect you've been meaning to pursue. Create a quarterly goals document for your team.

If you're a teacher or work in education: Build a unit plan for a topic you're teaching next. Create a parent communication template for the semester. Design a rubric for a project you've never been happy with.

If you're a parent or managing a household: Build a summer activity plan for your kids. Create a budget for a home improvement project you've been putting off. Plan a family vacation with an actual itinerary, not just a destination.

If you're a student: Build a study plan for your hardest class. Create a semester calendar with every deadline, exam, and project mapped out. Write a plan for landing an internship this summer.

If you're a creator or have a side project: Build a launch plan for something you've been working on. Create a content calendar for the next three months. Write a pitch for a collaboration you want to propose.

If you're starting over or figuring things out: Build a career change plan with specific steps and a timeline. Create a plan to learn a new skill in 90 days. Map out what you'd need to do to move to a new city.

Pick one. Open Claude. Start talking.

Quick Reference Card

From blank page to finished project — the process

Step	What you do	What it teaches Claude
Brain dump	Tell Claude everything — goals, constraints, obstacles, reality	Context: the more you share, the better the output
First structure	Read what comes back — don't respond yet	Claude organizes your chaos into a starting point
Push back	Tell it what won't work and why	Refinement: eliminate dead ends Claude can't see
Get specific	Ask for dates, numbers, daily actions — not categories	Specificity: the level of detail you ask for is the level you get
Stress test	Ask Claude to poke holes in the plan	Critique: find what breaks before you start
Polish	Ask for a clean final version, organized the way you'll use it	Finish: turn the conversation into a deliverable

Prompts worth stealing:

Here's everything in my head about [project]. [Dump it all — goals, situation, constraints, timeline, budget, obstacles.]

This won't work because [specific reason]. Rebuild this section with that constraint in mind.

Make it specific. Give me a week-by-week plan with daily actions I can check off.

Look at this plan as a skeptic. What's most likely to fail? Where am I being unrealistic?

Give me a clean final version I can print and follow. Start with the goal, then the plan, then what to track.

Chapter 3: Trust but verify

The Story

While writing my cybersecurity book, I needed an appendix covering global AI security regulations. There are more than a thousand of them worldwide, so I asked Claude to research the landscape and help me build a comprehensive list.

We went through multiple iterations. The first pass was too broad — hundreds of entries, most of them irrelevant to my audience. So, we narrowed it down. Which regulations would my readers, mostly security and risk professionals at mid-to-large companies, actually need to deal with? Claude filtered, organized, and summarized. After several rounds, I had a tight list of the most significant regulations by region.

I was reading through the final version when something caught my eye. One of the EU entries didn't have quite the same context as the others. The language was slightly off — not wrong exactly, but it didn't sit the way the real ones did. I couldn't point to a specific word that was the problem. It just felt like it didn't belong.

So, I asked Claude a simple question: can you cite a source for this regulation, or did you make it up?

Complete fabrication. The whole thing. Not a real regulation with the wrong details — an invented regulation that never existed. Claude had created a fake EU regulation, gave it a plausible-sounding name, described its scope and requirements, and slotted it right in between real ones. If I hadn't stopped to question it, it would have gone into my book.

That was the moment I realized the most dangerous thing about AI hallucinations isn't that they're obviously wrong. It's that they're mostly true, or at least they sound believable enough that you might not check. Claude didn't fabricate something ridiculous. It fabricated something that fit perfectly into the context of everything else on the list. That's what makes it dangerous — it blends in.

After that, I built a rule into every research prompt and every writing prompt I use: do not fabricate information. Do not invent statistics, names, dates, regulations, citations, or case studies. If you don't have the information, say so.

It still happens. Just not as often. And now I always check.

That's what this chapter is about. Claude is a remarkably capable tool — you've already seen that. But capable tools deserve capable operators. This is the shortest chapter in the book, and its lesson is simple: check the work that matters.

— — —

Build It: Breaking Claude on Purpose

You're going to do two exercises. The first one uses something you already know. The second one teaches you how to verify things you don't know. By the end, you'll have a personal verification checklist you can use every time you work with Claude.

Exercise 1: Test Claude on Your Turf

Think of a subject you know deeply. Your profession, your hobby, your area of expertise — something where you'd spot a wrong answer immediately. A nurse would know drug interactions. A mechanic would know engine specs. A teacher would know curriculum standards. A poker player would know hand rankings and odds.

Open Claude and ask it detailed questions about your subject. Don't ask easy ones. Push into the specifics — the kind of details where being close isn't good enough.

If you're an accountant, ask about a specific tax provision and its exceptions. If you're a contractor, ask about building code requirements for your state. If you're a musician, ask about the chord progression in a specific song.

Here's what you're looking for:

Tell me about [specific technical detail in your field]. Be precise — include dates, numbers, and specific requirements.

Read the response carefully. Not for whether it sounds smart — it will sound smart. Read for whether it's right. Check the details against what you know. Are the numbers accurate? Are the dates correct? Did it describe the process the way it actually works?

You might find everything checks out. That happens — Claude handles a lot of domain questions well. But push hard enough and you'll likely find a detail that's off. Maybe a number is close but not current. Maybe it described a process that's right in general but not quite how it works in your specific context. Maybe it confidently stated something that's debatable in your field.

If you find an error, don't just move on. Tell Claude:

That's not right. [Explain what's wrong.] Can you correct this and tell me where you got the original information?

Watch what happens. Claude will correct itself, usually with an apology. But here's the part that matters — ask it where the original wrong answer came from. Sometimes it will admit it doesn't have a reliable source. That's Claude being honest about its own limitations, and it's information you can use.

Exercise 2: Verify What You Don't Know

The first exercise was easy because you already knew the answers. This one is harder — and more realistic. Most of the time you're using Claude, you're asking about things you don't already know. That's the whole point. So, how do you verify something when you can't spot the errors yourself?

Ask Claude to research something you're genuinely curious about but don't have expertise in. It could be anything — a historical event, a scientific concept, a legal question, an industry you're not part of.

Research [topic] and give me the key facts. Include specific statistics, dates, and cite your sources where possible.

Now do something most people skip: verify it.

Pick three specific claims from Claude's response — a statistic, a date, and a name or citation. Look them up yourself. Use Google, use Wikipedia, use whatever source makes sense for the topic. You're not trying to become an expert. You're checking whether Claude's specific claims hold up.

Here's the prompt that changed how I work with Claude:

Did you fabricate any of the information in your last response? Be honest.

Ask it. You might be surprised by the answer. I was, and more than once. Claude will sometimes tell you outright that it's not confident about certain details, or that it reconstructed something from patterns rather than citing a specific source.

This is not Claude being broken. This is how AI works. It generates text that's statistically likely to be correct, but it doesn't know things the way you know things. It doesn't look up a fact and report it. It predicts what the right answer probably sounds like. Most of the time, that prediction is accurate. Sometimes it isn't. And it has no way to tell you which is which unless you ask.

Build Your Verification Checklist

Now that you've seen what can go wrong, build your own checklist. Open Claude and say:

Based on what we just did, help me build a personal verification checklist I can use every time I work with AI. It should cover the most common types of errors and the fastest ways to catch them.

Claude will draft something. Push back on it. Add the things you noticed in the two exercises you just did. Make it specific to the kind of work you do.

Here's the one I use. Yours will be different, but this is a starting point:

Before I trust any AI output, I check:

Does anything feel off, even if I can't say why? My gut has caught things my eyes missed.

Are the specific numbers, dates, and names verifiable? If I can't verify a claim in sixty seconds, I flag it.

Did I ask Claude if it fabricated anything? I ask this on every research task now. Every single one.

Is this going somewhere with consequences? An email draft has lower stakes than a book appendix or a client deliverable. The higher the stakes, the more I verify.

Would I bet money on this being right? If the answer is no, I check.

Print your checklist. Pin it next to your monitor. The point isn't to memorize it — it's to build the habit of pausing before you trust.

What Just Happened

Here's the thing about Claude: it's right far more often than it's wrong. The exercises you just did might have turned up errors, or they might not have. Either way, you now know how to check — and that's the skill that matters.

The reason verification matters isn't that Claude is unreliable. It's that when it does get something wrong, it sounds exactly like it does when it gets something right. There's no change in tone, no hedge, no yellow flag. It delivers everything with the same confidence. That means the occasional error blends in with all the accurate work around it. That's precisely why a lightweight check is worth the few seconds it takes.

In my experience, the people who get the most out of Claude are the ones who use it aggressively and verify deliberately. They don't treat every response like a suspicious document, but they do verify the things that matter.

The verification habit doesn't slow you down as much as you'd think. Asking "did you fabricate anything?" takes five seconds. Spot-checking a statistic takes thirty. That's a small investment for the confidence that what you're building on is solid.

The rule I built into my prompts — do not fabricate information, if you don't know, say so — made a real difference. Claude became more likely to tell me when it wasn't sure, which is almost more valuable than getting the right answer in the first place.

One thing I wish someone had told me early on: the errors aren't random. Claude tends to fill in gaps when you ask for very specific details — exact statistics, precise dates, niche citations. When you ask for general explanations or well-known information, it's usually solid. Knowing where the risk is highest lets you focus your verification where it counts, instead of second-guessing everything.

Now Go Make Something

You have a verification checklist. Now stress-test it on something real.

Go back to the project plan you built in Chapter 2. Read through it with fresh eyes. Are there any claims, numbers, or assumptions that came from Claude rather than from you? Verify them. Ask

Claude if it fabricated any of the specifics. You might find your plan is rock-solid. You might find a number that needs adjusting. Either way, you'll trust the plan more after you've checked it.

Then try this with something new:

If you work in a regulated industry: Ask Claude about a specific regulation that applies to your work. Check every detail against the actual source document.

If you manage budgets or finances: Ask Claude to estimate costs for something in your industry. Verify the numbers against real quotes or published data.

If you create content: Ask Claude to write something with specific facts and citations. Verify every citation exists and says what Claude claims it says.

If you're a student: Ask Claude to explain a concept from your hardest class. Compare its explanation to your textbook or lecture notes.

If you're planning something important: Ask Claude to research a key decision — a major purchase, a career move, a business strategy. Fact-check the three most important claims before you act on them.

The goal isn't to catch Claude being wrong. The goal is to build the muscle memory of checking. Once that habit is automatic, you'll be able to use Claude with confidence on everything that comes next — and there's a lot coming next.

— — —

Quick Reference Card

Trust but verify — your verification toolkit

Check	When to use it	How long it takes
"Did you fabricate anything?"	Every research task	5 seconds
Spot-check one specific claim	Any output with numbers or dates	30 seconds
Ask for sources	Anything going into a deliverable	1 minute
Cross-reference with a second source	High-stakes decisions or publications	5 minutes
Expert review	Anything with legal, medical, or financial consequences	Varies

The danger zones — where Claude is most likely to fabricate:

Specific statistics and percentages. Exact dates for niche events. Citations and source references. Names of obscure people, laws, or organizations. Details about very recent events.

The safe zones — where Claude is usually reliable:

General explanations of well-known concepts. Widely documented historical events. Common processes and how-to instructions. Brainstorming and idea generation. Writing and editing your own content.

The rule to add to every research prompt:

Do not fabricate information. Do not invent statistics, names, dates, or citations. If you don't have reliable information, say so

Chapter 4: Go deep

The Story

I'd been using Claude for research since the early days of my cybersecurity book. The process worked — I'd describe what I was looking for, ask specific questions, give Claude the context it needed, and we'd go back and forth until I had what I wanted. That's how the CEO presentation from Chapter 2 came together. I gave Claude sixteen pages of compiled research, and we distilled it down to three slides and a ten-minute talk. It worked because I was in the loop the whole time, steering the conversation, pushing back, refining.

But one day while working on this book, I asked Claude a simple question: is there a better way to do this?

I'd been spending a lot of time on the research phase of projects — feeding Claude context, asking follow-up questions, compiling the results, going back for more. It was infinitely faster than doing it manually, but I was still the one managing the whole process. Every search, every follow-up, every "now look at it from this angle" — that was me.

Claude told me about a feature I'd never noticed. A Research toggle, buried in the "+" menu at the bottom of the chat input. I'd clicked that menu dozens of times to attach files. I'd never seen it — or if I had, I didn't know what it did.

Here's the thing about working with any tool: you can use it productively for months and still not know about something that would change how you work. I'd been using Claude almost every day. I'd written an entire book with it. And I didn't know this feature existed.

There was a catch, though. I'd been working in Cowork mode — Claude's desktop mode where it can read your files, run code, and work alongside you. The Research toggle isn't there. I looked. It's not in Cowork. But it is in the Claude desktop app under Chat mode, and it's on the web version at claude.ai.

So, I opened a new chat, clicked the "+" button, and toggled on Research. Then I gave it a real question. Something I actually needed. I was getting ready to launch *Cyber Risk Is Business Risk* and I had no marketing plan. I didn't know how to promote a self-published book. I didn't know what services were worth paying for. I didn't know what I didn't know.

I typed out my situation: self-published on KDP with my own ISBN under a publishing brand I'd created. All marketing and promotion on me. Active on LinkedIn and Facebook. No TikTok, no Twitter. I needed a launch plan — free and paid options, specific services, a prioritized roadmap.

Then I hit Enter and watched.

This was different from anything I'd done with Claude before. Instead of getting a response in a few seconds, Claude went away. I could see it working — searching the web, following threads from one source to the next, building on what it found. It wasn't asking me for clarification. It wasn't waiting for me to steer. It was doing the research on its own.

Six minutes and forty-nine seconds later, it came back. Two hundred and eighty-seven sources. A complete launch playbook tailored to my specific situation — not generic advice for any author, but recommendations built around the fact that I was a cybersecurity expert self-publishing under my own newly created (with Claude) brand with an existing social media presence on LinkedIn, Facebook, and Instagram.

It told me to skip TikTok and Twitter entirely — which matched my instinct, but I'd been second-guessing. It gave me specific podcasts to pitch, by name, with host names and how to reach them. It recommended pricing ranges for Kindle and paperback with the reasoning behind each number. It laid out a staged action plan — what to do now, what to do at launch, what to do after — with benchmarks that would tell me when to change course.

I'd been using Claude for research for months. I thought I was good at it. But in under seven minutes, the Research feature did what would have taken me an afternoon of back-and-forth conversation — and it went wider than I would have thought to go.

That's what this chapter is about. You already know how to have a conversation with Claude. Now you're going to learn how to send it on a mission.

Build It: Run Your First Research Investigation

You're going to use Claude's Research feature to investigate a real question — something you actually need answered. By the end, you'll have a comprehensive, cited report you can act on.

Where to Find It

Research is available in two places: on claude.ai (the web version) and in the Claude desktop app under Chat mode or in the mobile version. It's not available in Cowork mode. If you've been working in Cowork, you'll need to switch to Chat, use the mobile app or open claude.ai in your browser.

Here's how to get to it:

Open a new chat. Look at the bottom of the input area where you type your messages. You'll see a "+" button. Click it. A menu will pop up with several options — file uploads, skills, connectors. Near the bottom of that menu, you'll see **Research**. Click it.

You'll see an indicator that Research is now active. That's it — you're ready.

One thing before you start: make sure **Web search** is also toggled on in that same menu. Research needs web search to work. If web search is already on (you'll see a checkmark next to it), you're good.

Pick Your Question

This is the most important step, and most people get it wrong. They ask something broad — "tell me about marketing" or "what's the best way to start a business." That's like hiring a research analyst and saying "go learn about stuff."

The Research feature works best when you give it what you'd give a smart person you were sending off to do a day's worth of research for you:

What specifically you want to know. Why you want to know it. What you plan to do with the information, and the details about your situation that make your question different from anyone else's.

Here are some options. Pick whichever one is real for you or drop in your own question entirely.

If you run a business: "I'm considering switching from [current tool] to [new tool] for my team of [size]. Research the full picture. Am I even looking at the best tool? Are there other options I haven't considered? Give me the cost comparison, migration risks, what companies similar to mine have experienced, and whether the switch is worth it and why."

If you have an existing website or brand: "I would like to maximize the appeal and effectiveness of my brand. Look at the top-rated websites in my industry and their branding. Tell me what is missing from mine and what I should consider changing. Tell me what free and paid tools are available to help me build my brand and which ones I should consider using."

If you're in a regulated industry: "I need to understand how [specific regulation] affects [your specific business type]. What are the compliance requirements, what are companies getting wrong, and what does a realistic compliance roadmap look like for a company our size?"

If you're a teacher or in education: "I want to redesign my [subject] curriculum to incorporate AI tools. What are schools actually doing that's working? What are the risks? How are teachers innovating and engaging students by using AI to enhance the learning experience? Give me specific examples and programs I can look at. If they don't exist, maybe I should create my own."

If you're making a major decision: "I'm considering [specific decision — buying a home in a specific area, changing careers, starting a side business]. Research everything I'd need to know to make this decision well. Include what people in my situation commonly get wrong."

If you're just curious about something: Pick a question you've been meaning to dig into but never had the time. The kind of thing you'd spend a weekend reading about if weekends were longer.

Write Your Prompt

Don't just type the question. Give Claude the context it needs to bring back something useful. Think of it as a briefing for your research assistant. That is exactly what you are doing. You are sending Claude on an assignment, and the better the briefing, the better the report. The difference is, you are paying Anthropic less than you would have to pay your research assistant.

Here's the structure that works:

I want you to research [your question]. Here's the context: [your situation, your constraints, what makes your case specific]. I plan to use this research to [what you'll do with it]. I need [what the output should look like — a comparison, a recommendation, a prioritized list, a decision framework].

The more specific you are about your situation, the more tailored the results will be. When I asked about my book launch, I didn't just say "how do I market a book." I told Claude it was self-published on KDP, under my own imprint, that I was active on LinkedIn but not on TikTok, and that marketing was entirely on me. Every one of those details shaped the recommendations I got back.

Hit Enter and Wait

This is the part that feels different. With a normal Claude conversation, you get a response in seconds. With Research, Claude goes away for a while. You'll see it working — searching, reading sources, following threads.

Don't close the tab. Don't get impatient. I'll admit I almost did — six minutes feels like an eternity when you're used to getting a response in five seconds. But the feature is doing something genuinely different from a regular response. It's searching the web across dozens — sometimes hundreds — of sources, reading them, following leads from one to the next, building a picture before it writes a word. Think of it less like asking a question and more like dispatching a very fast intern to the library.

My book launch research took six minutes and forty-nine seconds and hit 287 sources. Yours might take three minutes and hit fifty. It depends on the complexity of the question. Either way, let it run.

Read What Comes Back

When the report arrives, resist the urge to skim it. Read it the way you'd read a report from a consultant you hired. Look for several things:

Structure. Is it organized in a way that makes sense for your question? A good research report has a clear summary at the top, detailed findings in the middle, and actionable recommendations at the end.

Specificity. Are the recommendations specific to your situation, or could they apply to anyone? If you gave Claude good context, the results should feel tailored. If they feel generic, you may not have given enough detail in your prompt.

Citations. Research reports come with sources. Click a few of them. Do they actually support what Claude is claiming? This is the verification skill you built in Chapter 3 — apply it here.

Surprises and gaps. The most valuable part of a research report is usually something you didn't think to ask about. When I got my book launch playbook, I didn't know about specific podcasts in my niche, or that HARO had been acquired and replaced, or that KDP Select was probably wrong for business nonfiction. Those surprises are where the real value is. But also notice what's missing — no research is exhaustive. If something important isn't covered, ask follow-up questions in the same conversation. Claude has the full context of what it just researched.

Make It Actionable

A research report sitting in a chat window doesn't do anything. You need to turn it into something you'll actually use.

Tell Claude:

Take the key recommendations from this research and turn them into a prioritized action plan. Group them by timeframe — what I should do this week, this month, and this quarter. For each action, tell me the first concrete step.

You can also ask Claude to pull out specific sections:

Give me just the free options, ranked by impact.

Which of these recommendations is most important for someone in my specific situation?

What's the one thing from this research I should do first?

Save the results somewhere you'll actually look at them. Copy the report into a document. Print the action plan. Put it where your project plans live — not buried in a chat history.

What Just Happened

You just outsourced a research project. Not to a person who would take a week and charge you for it, but to a tool that did it in minutes.

Here's what I want you to notice about the difference between this and what you did in Chapter 2.

In Chapter 2, you were the engine. You gave Claude context, read what came back, pushed back on it, asked for more detail, steered the conversation toward what you needed. The output was excellent because you were in the loop the whole time, making judgment calls at every step.

With Research, Claude is the engine. You gave it a briefing and it went off on its own — decided what to search for, which sources to read, which threads to follow, how to organize what it found. You weren't in the loop during the investigation. You got the finished report.

Both approaches work. They're for different situations.

Use the conversational approach that you've been using when you already have the source material, when you need to iterate on something specific, or your judgment at each step is what makes the next iteration of the output better. The CEO presentation was perfect for this. I had the research. I needed Claude's help organizing and distilling it.

Use Research when you need to go wide. When you don't know what you don't know. When the question is big enough that manually searching source after source would eat your afternoon. When you want Claude to find things you wouldn't have thought to look for.

In my experience, the best results come from combining both. Run Research first to get the landscape. Then take the results and work with them conversationally — pushing back, asking follow-ups, applying your own judgment to the recommendations. The first pass gives you breadth. The conversation gives you depth.

One more thing worth knowing: Research reports come with citations, but they're not perfect. The same verification habits from Chapter 3 apply here. If you're going to act on a specific recommendation — especially one involving money or legal decisions — verify it. The report is a starting point, not a finished answer. A very good starting point, but a starting point.

Now Go Make Something

You've run one research investigation. Now run another — on something completely different.

The goal isn't to become a Research power user. The goal is to build the instinct for knowing when a question deserves a real investigation rather than a quick conversation.

Here are some ideas. Pick one that's real for you.

If you manage people or run a team: Research the best practices for something your team struggles with — onboarding, remote work policies, performance reviews. Ask for what companies your size are actually doing, not what consultants recommend.

If you're making a financial decision: Research a major purchase or investment you're considering. A home in a specific neighborhood. A car. A renovation. Ask for comparable data, common mistakes, and what people who've done it wish they'd known.

If you work in a specialized field: Pick a question you've been meaning to research but never had time for. A new regulation, a competitor analysis, a technology you've been hearing about. Give Claude your professional context and let it go deep.

If you're a student: Research graduate programs, career paths, or industries you're interested in. Ask for data, not opinions — acceptance rates, salary ranges, day-in-the-life accounts, what people in those roles actually do.

If you have a personal project: Research how to do something you've been putting off — learning an instrument, training for an event, planning a trip, starting a hobby. Ask for specific programs, resources, and realistic timelines.

After you get the report, do one more thing: take the results back into a regular Claude conversation and work with them. Push back on the recommendations. Ask which ones matter most for your situation. Turn the research into a plan with specific next steps.

That combination — Research for breadth, conversation for depth — is how you get the most out of Claude's capabilities. And the fact that you now know both approaches means you can pick the right tool for the question, every time.

Quick Reference Card

Research — your investigation toolkit

Feature	Where to find it	What it does	When to use it
Research	claude.ai or Claude Desktop (Chat mode) — click "+" in the input area	Autonomously searches hundreds of sources and compiles a cited report	Big questions, unknown unknowns, going wide on a topic
Web search	Same "+" menu	Claude can look things up during a conversation	Quick fact-checks, current information, specific lookups
Conversational research	Any Claude chat	You and Claude go back and forth, building the output together	When you have the source material, when you need to iterate, when your judgment drives the output

How to write a good Research prompt:

I want you to research [specific question]. Here's my situation: [your context — role, industry, constraints, what makes your case unique]. I plan to use this to [what you'll do with it]. Give me [what format you want — recommendations, comparison, action plan].

What to look for in the report:

Is it tailored to my situation or generic? Are the citations real? (Click a few.) What surprised me — what didn't I think to ask about? What's missing that I still need to know? What's the first thing I should actually do?

The combination that works best:

Research first (go wide) → Conversation second (go deep). Run the investigation, then take the results into a regular chat and push back, ask follow-ups, and turn findings into action.

Where Research is and isn't available:

Available on claude.ai (web), Claude Desktop under Chat mode, and Claude Mobile. Not available in Cowork mode. Requires a paid Claude plan and Web search to

Chapter 5: See it

The Story

I had been working on a tracker for a new campaign we were launching. This had nothing to do with research or writing — this was a build-something-I-can-see kind of problem. We needed a way to manually track specific activities and new opportunities as they came in — something my team could open, update, and share without any special tools. Just a clean Excel spreadsheet in a shared Teams folder.

My creative brain was not working at its best that day. I'd been staring at a blank spreadsheet for longer than I'd like to admit, trying to figure out the right columns, the right layout, the right way to organize the data so it would actually be useful six weeks into the campaign when we had real numbers to look at. I knew what I wanted. I just couldn't get it out of my head and onto the screen.

Then it occurred to me that Claude might be able to help.

I dropped in my raw data — the rough notes I'd been keeping about the campaign, the kinds of activities we wanted to track, the metrics that mattered — and described what I needed. A tracker that could handle these specific data points, organized in a way that made sense for a sales team, with some basic structure so we weren't just dumping numbers into random cells.

Less than ten minutes and three iterations later, I had something I could post in a shared Teams folder for my team to use. It was a fully functional tracker with the right columns, sensible formatting, and a layout that actually worked for what we needed.

Three things struck me about that experience. First, I went from stuck to done in under ten minutes. I'd already spent longer than that staring at a blank spreadsheet getting nowhere. Second, the iterations were fast — I'd say "move this here, add a column for that, change how this section is organized" and Claude would rebuild the whole thing in seconds. Third, and this is the part that surprised me most, Claude didn't just do what I asked. On the second iteration, it suggested a way to organize the opportunity pipeline that I hadn't thought of. It was better than what I'd asked for.

That was the moment I started thinking differently about what Claude could create. I'd been using it for writing and research — words on a page. But Claude can also build things you can see, interact with, and share. Charts from your data. Flowcharts of your processes. Interactive tools — calculators, decision matrices, quizzes — that work right inside the conversation, plus dashboards that pull it all together. And it does all of this from a description. You don't need to know how to code. You don't need design software. You describe what you want, and Claude builds it.

That's what this chapter is about. You're going to create something visual — something you can look at, not just read.

Build It: Create Something You Can See

You're going to build two things in this chapter. The first is a visual that lives inside Claude — an artifact you can interact with right in the conversation. The second is a file you can take with you — a chart, a spreadsheet, a document with real visual structure. Both start the same way: you describe what you want.

Part 1: Build an Interactive Artifact

Claude can create visual, interactive things that appear right in the chat window. These are called artifacts — charts, flowcharts, calculators, decision tools, even simple games. They render live, right there, and you can interact with them without leaving the conversation.

Let's build one. Pick a scenario that's real for you:

If you manage a process: Think of a workflow your team follows — how a customer request gets handled, how a project moves from idea to delivery, how a new hire gets onboarded. You're going to turn that process into a visual flowchart.

If you work with numbers: Think of data you have — sales figures, budget numbers, survey results, grades, anything with numbers that tell a story. You're going to turn that data into a chart.

If you make decisions: Think of a decision you or your team makes repeatedly — which vendor to choose, which candidate to hire, which project to prioritize. You're going to build a decision tool that walks through the criteria.

Pick one. Here's how to start:

I manage [describe your process/data/decision]. I want you to create a visual [flowchart/chart/decision tool] that shows [what you want to see]. Here's what the process looks like: [describe the steps, the data, or the criteria].

Let me show you what this looks like in practice. Say you run a small business and you want to visualize your customer journey — from first contact to closed deal. You'd say something like:

I run a home renovation business. When a potential customer contacts us, here's what happens: they fill out a form on our website or call us. We schedule a site visit within 48 hours. After the visit, we send a quote within 3 business days. If they accept, we schedule the job and collect a deposit. If they don't respond within a week, we follow up once. If they decline, we ask why and log it. Create a flowchart that shows this process so I can share it with my team.

Hit Enter. Claude will generate a visual flowchart — boxes, arrows, decision points — right in the conversation. You can see it immediately.

Now here's the step most people skip. Look at what came back and tell Claude what's wrong.

The flow is right but I want it to show where we lose most customers. Can you add color coding — green for steps that usually go well, yellow for where deals stall, and red for where we lose people? Also, the quote step should show that we have three different quote templates depending on the job size.

Claude rebuilds it. The flowchart now has color, detail, and structure that matches your actual business — not a generic process diagram you'd find in a textbook.

One more round. Push it further:

This is good. Can you add the average time each step takes? And I want a box at the bottom that shows our conversion rate at each stage — what percentage of leads make it from one step to the next.

Now you have something genuinely useful. A visual process map with timing data and conversion metrics that you could print, share in a meeting, or post in your office. You built it in minutes by describing it in plain English. And if you want to keep it, you can download the artifact or copy the code behind it — it doesn't have to live only inside the chat.

Part 2: Build Something You Can Take With You

Artifacts are useful, but sometimes you need a file — something you can save, email, open in Excel, or drop into a shared folder. This is where Claude creates actual documents with visual structure.

Think about something you need for your work or life. Pick one:

A chart or dashboard from real data. If you have numbers — sales, expenses, survey results, grades, workout logs, anything — you can give them to Claude and ask for a visual breakdown.

A formatted tracker or planner. Like the campaign tracker I built, but for whatever you need to track — a project, a budget, a hiring pipeline, a content calendar.

A visual report. Something with charts, tables, and formatted sections that you could hand to your boss or share with your team.

A PowerPoint presentation. You did your research and consolidated your message for your audience and time slot. Let Claude take the first shot at building your slides? Give it another deck that is on brand or give it your brand guide and tell it what you want and why. We'll go deeper on presentations in the next chapter.

Here's how to do it. Let's say you want to build a budget tracker for a project you're managing:

I'm managing a kitchen renovation with a total budget of \$45,000. Here are my categories and estimated costs: cabinets \$12,000, countertops \$5,000, appliances \$8,000, labor \$15,000, plumbing \$3,000, electrical \$2,000. I want an Excel spreadsheet that tracks estimated vs. actual costs, shows what percentage of the budget each category represents, and has a running total so I can see how much I've spent and how much is left. Make it something I'd

actually want to use — clean formatting, not cluttered.

Claude will build the spreadsheet. If you're in Cowork mode, it can save the file directly to your folder. If you're in Chat mode, you can download it.

Open the file. Look at it the way you'd evaluate work from a colleague. Is the layout clean? Do the formulas make sense? Is anything missing?

Then iterate:

Add a column for payment status — not started, deposit paid, in progress, paid in full. And add conditional formatting so that any category where actual costs exceed estimated costs shows up in red.

This is the same process you used in Chapter 2 — describe, review, push back, refine. The difference is that now you're building something visual. Something with structure and layout and formatting that you can see and interact with, not just read.

What If You Don't Have Data?

Don't let the lack of data stop you. In my experience, half the time the reason people don't have a tracker or a dashboard isn't that they don't have data — it's that they don't have a structure to put it in. Claude can build the structure first. You fill in the numbers as they come.

I want to start tracking my team's project pipeline. I don't have a system yet — everything is in emails and sticky notes. Can you create a project tracker template? We usually have 10-15 active projects at a time. I need to see the project name, client, status, deadline, who's responsible, and any blockers. Make it so I can sort by deadline or status.

Claude builds the template. You fill it in as you go. That's exactly what happened with my campaign tracker — I didn't have clean data when I started. I had notes and a rough idea of what I wanted to measure. Claude turned that into a structure, and the structure made it obvious what to track.

— — —

What Just Happened

You just built visual productivity tools by describing them. You were the artist with the vision. Claude brought it to life. It took care of the spreadsheet formulas, the formatting, and the visual aesthetics.

Here's what I want you to take away from this chapter.

The thing that stopped me from building that campaign tracker wasn't a lack of skill. I know how to use Excel. I've built dozens of trackers. I was stuck because the creative part of me that needed to figure out the right structure, the right layout, the right way to organize the information wasn't clicking that day. Claude got me past that wall in minutes.

That's what Claude does best with visual work. Claude isn't building things you couldn't build yourself, but it sure helps when you are staring at a blank screen trying to figure out where to start. You describe what you need in plain English, and Claude gives you something to react to. From there, it's iteration. Move this, change that, add a column, make it color-coded. Each round takes seconds.

The other thing worth noticing is how the process connects to everything you've already learned. The brain dump from Chapter 2 works here too — the more context you give about what you need and why, the better the visual output. The verification habit from Chapter 3 applies — check that the formulas work, that the flowchart logic is right, that the chart isn't misrepresenting your data. And the research-then-build pattern from Chapter 4 connects naturally — you could run a Research investigation to gather data and then build a visual from the results.

You're stacking skills, and each one makes the next one easier.

— — —

Now Go Make Something

You've built something visual. Now build something else — something you'll actually use this week.

Here are ideas. Pick one.

If you manage a team: Build a visual onboarding checklist for new hires. Or create a process flowchart for something your team does regularly that nobody's ever documented.

If you handle money: Build a personal budget dashboard. Or create a financial comparison for a big purchase you're considering — show the numbers side by side.

If you teach or train: Build a visual quiz or assessment. Or create a lesson timeline that shows how a class session flows from one activity to the next.

If you plan events or projects: Build a visual project timeline with milestones and dependencies. Or create a seating chart, a venue layout, or an event schedule.

If you create content: Build a content calendar with visual status indicators. Or create a visual brand guide — colors, fonts, tone descriptions — that you can share with collaborators.

If you're solving a personal problem: Build a decision matrix for something you've been going back and forth on. Lay out the options, the criteria, and the weights. Let the visual make the decision clearer.

Whatever you build, do it the way you've learned: describe what you want, look at what comes back, push back on what's wrong, iterate until it's right.

— — —

Quick Reference Card

See it — building visual things with Claude

What you can build	How to ask for it	Where it works
Flowcharts and process maps	"Create a flowchart showing how [process] works"	Chat, Cowork
Charts and graphs	"Here's my data: [data]. Create a [bar/line/pie] chart showing [what you want to see]"	Chat, Cowork
Interactive calculators	"Build a calculator that [computes what]"	Chat (artifacts)
Decision tools	"Create a decision matrix for [decision] with these criteria: [criteria]"	Chat (artifacts)
Spreadsheets and trackers	"Build an Excel spreadsheet that tracks [what] with columns for [details]"	Cowork (saves to folder), Chat (download)
Dashboards	"Create a dashboard showing [metrics] from this data: [data]"	Chat (artifacts), Cowork

The process (same as always):

Describe what you want and why → Look at what comes back → Tell Claude what's wrong → Iterate until it's right → Save it somewhere you'll use it

Tips for better visual output:

Give Claude your actual data or a realistic example. Tell Claude who the audience is — "my team," "my boss," "my students." Specify the format — "make it an Excel file," "show it as a flowchart," "build it as an interactive tool." Ask for color coding, formatting, or conditional logic if you want it. Don't stop after the first version.

When to use artifacts vs. files:

Artifacts (interactive, in the chat): Great for exploration, quick visuals, interactive tools, things you want to see and adjust quickly. Files (Exc

Chapter 6: Present it

The Story

One of our team had written a white paper that we could build a new service offering around. The research was solid, the recommendations were specific, and we had real data to back it up. Now we needed to sell it.

The problem was that we had two very different audiences we needed to test the messaging with. We wanted to pitch this to business stakeholders who could approve the budget if they saw the business value. We also wanted to pitch it to the technical teams who would actually implement and support what we were proposing. Both groups needed to hear the same message, but they needed to hear it in completely different ways.

Normally, I'd build one deck and adjust my talk track for each room. Same slides, different emphasis. But this wasn't that kind of pitch. The business audience cared about risk, cost, and market opportunity. The technical audience cared about architecture, integration, and whether what we were proposing would actually work in their environment. Same white paper, but the slides themselves needed to be in a different language and hit completely different priorities.

Building two distinct pitch decks from scratch would have taken the better part of a day, maybe longer, because I'd be fighting the urge to do what I always do — adjust the talk track, not the slides. Once you've framed an argument one way, it's hard to reframe it for a different audience without the first version bleeding through.

I decided to let Claude take a shot at it.

I gave Claude the white paper and explained the situation. Two audiences, two decks. For the business stakeholders, lead with the risk and the opportunity. What is the business benefit if they move forward, and what is the risk if they don't? For the technical team, lead with the architecture and the implementation. Show them how it works, what it integrates with, and what the migration path looks like. I told Claude I wanted both decks to be 10-12 slides each, with speaker notes I could use as a talk track.

Both versions landed surprisingly well on the first pass. Claude didn't just change the vocabulary — it restructured the argument. The business deck opened with a market trend and built toward a cost-of-inaction case. The technical deck opened with the current architecture gap and built toward a migration roadmap. Same underlying research, but the framing was so different that you wouldn't immediately recognize them as coming from the same source.

Both decks required additional iteration and refinement outside of Claude. The business deck needed sharper financial projections. The technical deck needed more detail on integration points. But Claude had done the heavy lifting — the structure, the story arc, the slide-by-slide logic. I was editing and refining, not building from scratch. It saved me hours.

That experience changed how I think about presentations. The hard part of building a deck isn't making slides. It's figuring out what to say to this specific audience and in what order. That's the part Claude is genuinely good at — taking your content and restructuring it for the room you're walking into.

Build It: Create a Presentation That Lands

You're going to build a real presentation in this chapter. Not a generic template — something you'd actually deliver. And then you're going to do something most people never do: reframe it for a completely different audience.

Part 1: Build Your Deck

Start with content you already have. It could be a report, a proposal, research you've done, notes from a project, or even the output from a previous chapter. If you built a project plan in Chapter 2 or ran a research investigation in Chapter 4, that's perfect source material for a presentation.

If you don't have existing content, pick something you actually need to present. A project update for your team. A proposal for your boss. A training session for new hires. A pitch for a client.

Here's how to start:

I need to build a presentation about [your topic]. My audience is [who they are — their role, what they care about, what they're skeptical of]. I have [time limit] to present. Here's the content I'm working from: [paste your content, notes, or key points]. Build me a slide deck with [number] slides, including speaker notes for each slide.

The more you tell Claude about your audience, the better the deck will be. "My audience is executives" gets you a generic business presentation. "My audience is the VP of Operations who has shot down the last two proposals from our department because they didn't include implementation timelines" gets you something targeted.

Let me show you what a good briefing looks like:

I'm presenting a quarterly update to my company's leadership team. I have 15 minutes. The audience is the CEO, CFO, and three VPs. They care about revenue impact and timeline — they don't want technical details. Last quarter we missed our delivery target by two weeks, so I need to address that without dwelling on it. Here are my key points: [your points]. Build a 10-slide deck with speaker notes. The first slide should hook them with the headline result before they start checking their phones.

Hit Enter. Claude will generate a complete deck outline with slide titles, content for each slide, and speaker notes.

Now do what you've been doing since Chapter 2. Look at what came back and refine it.

Slide 3 is too dense — split it into two slides. The speaker notes for slide 5 sound like I'm reading a report. Make them conversational — I want to sound like I'm talking, not presenting. And move the timeline slide earlier. They're going to be thinking about the missed deadline from last quarter, so I should address it in the first three minutes, not the last three.

Claude rebuilds. The deck gets sharper with each round because you're bringing the thing Claude doesn't have — knowledge of that specific room, those specific people, and what they need to hear.

One more round:

The closing slide is weak. I don't want to end with "next steps" — I want to end with the one number that makes the case. What's the strongest way to close this?

Now you have a deck structure with speaker notes that's tailored to your audience and your presentation style. From here, you can build the actual slides in PowerPoint, Google Slides, or whatever tool you use — or if you're in Cowork mode, you can ask Claude to generate the .pptx file directly.

Part 2: Reframe It for a Different Audience

This is the step that saved me hours with my pitch decks, and it's the one most people don't think to try.

Take the presentation you just built and reframe it for a completely different audience. Same core content, different room.

I just built this presentation for [audience A]. Now I need to present the same information to [audience B]. They care about [different things]. Rebuild the deck for this audience — don't just change the words, restructure the argument so it leads with what matters to them.

That last line is important. If you just ask Claude to "adjust the tone," you'll get the same deck with different adjectives. If you tell it to restructure the argument, you'll get a fundamentally different presentation — different opening, different evidence highlighted, different arc.

Here are some common reframes:

Executives → Technical team. The executive version leads with outcomes and cost. The technical version leads with how it works and what it takes to implement.

Internal team → External client. The internal version can be candid about problems and gaps. The external version frames the same information as capabilities and solutions.

Skeptical audience → Supportive audience. The skeptical version front-loads evidence and addresses objections early. The supportive version front-loads the vision and saves the details for the

appendix.

Expert audience → General audience. The expert version assumes vocabulary and jumps to what's new. The general version builds context first and uses analogy instead of jargon.

Try one. You'll be surprised at how different the two decks look — and how much faster it is to build the second one when Claude already has the source material in context.

Part 3: Build a Formatted Document

Presentations aren't the only professional deliverable Claude can build. Reports, proposals, memos, project briefs — anything with structure, formatting, and a specific audience can follow the same process.

Pick something you need:

I need a [report/proposal/brief] for [audience]. It should cover [topics]. Here's the background: [your content]. Format it professionally — I want section headers, an executive summary, and a recommendations section. Make it something I could send to [my boss / a client / my board] without being embarrassed.

The key phrase there is "without being embarrassed." That tells Claude you want professional polish, not a rough draft. The more specific you are about the format and audience, the closer the first version will be to what you need.

Then iterate the same way you've been doing all along. Push on the sections that feel generic. Ask Claude to tighten the executive summary. Tell it the recommendations need to be more specific — not "consider implementing" but "implement X by Q3 at an estimated cost of Y."

— — —

What Just Happened

You just built professional deliverables by describing what you needed and who needed to hear it. The whole thing was reframed for a different room — it all came from the same source material. You and Claude reframed it to view through a different lens.

Here's what I want you to take away.

The audience reframe is the thing that surprised me most. When I built those two pitch decks from the white paper, I expected Claude to swap some vocabulary and rearrange a few slides. Instead, it rebuilt the argument from the ground up. The business deck told a story about market risk. The technical deck told a story about architecture and integration. Same research, completely different narrative structure. This is beyond cosmetic. This is the kind of strategic thinking that usually takes hours of staring at a whiteboard.

In my experience, the reframe is where Claude adds the most value with presentations. Building a first draft of a deck is useful. Building a second version for a different audience in minutes is the real

win, because that second version is the one most people never build. They run out of time, so they walk into the wrong room with the right slides.

The other thing worth noting is how naturally this connects to everything you've built so far. The research from Chapter 4 becomes the content for your presentation. The visual artifacts from Chapter 5 become charts and diagrams in your slides. The iteration process from Chapter 2 is the same process you used to sharpen the deck. You are building on the foundation skills you already have.

Now Go Make Something

You've built a presentation and reframed it. Now build a deliverable you actually need this week.

Here are ideas. Pick one.

If you have a meeting coming up: Build the deck for it. Give Claude your notes, your agenda, and who's in the room. Walk in with slides and a talk track instead of winging it.

If you need to report on something: Build a formatted report with an executive summary, findings, and recommendations. Give Claude your raw data or notes and tell it who the report is for.

If you need to pitch something: Build the pitch deck. Then build a second version for the other decision-maker — the technical lead, the finance person, the skeptic who needs different evidence.

If you train or teach: Build a training deck. Give Claude your curriculum and tell it about your learners, their experience level, what they struggle with, and how long you have. Then reframe it for a different group.

If you need to document a process: Build a process document with flowcharts, step-by-step instructions, and a quick-reference summary. Ask Claude to make it something a new hire could follow on their first day.

If you're working on this book's exercises: Take the project plan from Chapter 2 or the research report from Chapter 4 and turn it into a presentation. Pick an audience — your team, your boss, your board — and build slides that would land in that room.

Whatever you build, remember the reframe. One presentation is useful. Two versions of the same presentation, each tailored to a different audience, is your advantage.

Quick Reference Card

Present it — building deliverables with Claude

What you can build	How to ask for it	Key details to include
--------------------	-------------------	------------------------

Slide deck	"Build a [number]-slide deck about [topic] with speaker notes"	Audience, time limit, what they care about, what they're skeptical of
Reframed deck	"Rebuild this for [different audience] — restructure the argument, don't just change the words"	What the new audience cares about, how they're different from the first
Formatted report	"Write a [report/proposal/brief] for [audience] with an executive summary and recommendations"	Tone, format expectations, what decisions the reader is making
Project proposal	"Build a proposal for [project] that covers scope, timeline, cost, and risks"	Who approves it, their concerns, budget constraints
Training materials	"Create a training deck for [topic] aimed at [learners] for a [duration] session"	Experience level, learning objectives, what they struggle with

The reframe — turning one deck into two:

I built this presentation for [audience A]. Now rebuild it for [audience B]. They care about [different priorities]. Don't just adjust the tone — restructure the argument so it leads with what matters to them.

What makes a good presentation briefing:

Tell Claude who's in the room. Tell Claude what they care about and what makes them skeptical. Tell Claude how much time you have. Tell Claude what happened last time — the context they're walking in with. Give Claude your content, not just your topic.

The process:

Give Claude your content and audience → Get the first deck → Push back on structure, tone, speaker notes → Reframe for a second audience → Export and polish in your presentation t

Chapter 7: Claude on your computer

The Story

By this point, I had too many active projects to keep straight in my head.

I was using Claude for everything. I was working on three books, a couple of app projects with Chuck, work stuff, and continuing to brainstorm for new ideas. Everything up to this point had been conversations inside the desktop app. Every conversation lived in the same memory space, and the more I added, the longer things took and the more things started to cross connect. Claude was looking at everything when I only needed it focused on what I was working on right now.

I'd done some training through Anthropic's learning resources, and while Chuck had gotten me up and running on my own from day one, the training filled in knowledge gaps I didn't know I had. One of the things I learned about was Cowork Projects.

The idea is simple. Instead of every conversation sharing one big pool of context, you create separate projects — each with its own folder, its own files, and its own set of rules. When I'm working on this book, Claude sees the manuscript files, the drafting rules, the framework document, and nothing else. When I switch to one of the app projects, Claude sees that codebase and those requirements. There is no bleed over or confusion. I don't waste time sifting through context that doesn't apply.

I set up a projects folder on my machine with subfolders for every active workstream. Then I had Claude help me package up everything in chat related to each project and move it into the right folder. It took some effort — sorting through conversation history, pulling out the files and rules that belonged to each project, making sure nothing got lost in the migration. But it was totally worth it.

The difference was immediate. Claude was faster because it wasn't searching through irrelevant context. The responses were more focused because the project-specific rules I'd built — voice guidelines for the book, technical constraints for the apps — all stayed where they belonged. Each project had its own workspace, and Claude behaved like a different collaborator depending on which one I opened.

The next thing I did was have Claude build a project plan for each active project. With all the files in one place and the scope clearly defined, Claude could see the whole picture and help me lay out what needed to happen, in what order, and by when. I ended up with structured plans for every workstream, each one tailored to what that project actually needed.

Then I tried something that changed how I start every morning.

I built a scheduled task — a daily check-in that runs automatically. Every morning, Claude looks at the progress from the previous day, lays out the work plan for today, and gives me an overview of where I stand across everything. It includes the schedule, an updated time-to-complete estimate that adjusts based on how much I actually get done each day, and it calls out upcoming deadlines and milestones. It adjusts the plan based on how much time I tell it I can commit, then reconciles what I actually did.

The daily check-in is the single most useful thing I've built with Claude. It turned Claude from a tool I talk to into something closer to a personal assistant that knows my projects, tracks my progress, and shows up every morning with a plan before I've finished my first cup of coffee.

Build It: Give Claude a Workspace

Everything you've built so far has been in individual conversations in chat. That works fine when you're doing one thing at a time. But the moment you're juggling more than one project — and you will be — you need Claude to focus. That's what projects do.

You're going to set up a real project, put Claude to work on your actual files, and then set up something that runs on its own. By the end of this chapter, Claude won't just be a tool you open when you need something. It'll be working alongside you.

Part 1: Create Your First Project

You already have the Claude desktop app — you've been using it since Chapter 1. Now you're going to give it a dedicated workspace.

Think of a project you're actively working on. It could be anything — a work initiative, a side business, a creative project, a class you're taking or home renovations you're planning. The only thing that changes is everything you have related to this project gets stored in the project folder. That includes all of the documents, notes, plans, and emails you've saved. Anything and everything you have related to the project goes into the space.

First, organize the files. Create a folder for the project if you don't already have one. Put everything relevant in it. Don't overthink this — you can always add more later. The point is to give Claude a place to look.

Now create a new project in Claude. Give it a name that means something to you — not "Project 1" but "Kitchen Renovation" or "Q3 Marketing Plan" or "Side Business Launch." Then connect your folder. Claude will ask which folder to use. Point it at the one you just set up. I created a cowork projects folder in my documents folder in windows. It doesn't matter where you put it, as long as you can navigate to it.

Here's the step that makes projects powerful. Set up project instructions — the rules Claude should follow when working in this project. Think of it as briefing a new team member on their first day. What

should they know? What tone should they use? What matters most?

This project is [describe it in two sentences]. The audience for everything we create is [who]. When I ask you to write, use [tone — professional, casual, technical, plain English]. The most important files in this folder are [list them]. Here's what we're trying to accomplish: [the goal].

Those instructions stay with the project. Every conversation you open inside it starts with Claude already knowing the context. You won't have to re-explain what you're working on or re-establish the rules. That's the whole point.

Part 2: Put Claude to Work on Your Files

Now that Claude can see your folder, put it to work.

Start with something simple. Ask Claude to read what's there and tell you what it sees.

Look at the files in my project folder. Give me a summary of what's there — what each file is, what it covers, and what's missing.

This does two things. It confirms Claude can actually access your files, and it gives you an outside perspective on your project's current state. You might be surprised by what Claude notices — gaps you've been too close to see, files that overlap, information that's scattered across three documents when it should be in one.

Now ask Claude to create something from what it found.

Based on what you see in these files, build me a [project status summary / consolidated brief / action item list / organized outline]. Pull from the actual content — don't make things up.

This is where it starts to feel different from chatting. Claude isn't generating content from scratch — it's reading your actual work and synthesizing it. The project plan you built in Chapter 2 came from your brain dump. This comes from files you've already created over days or weeks or months.

Push it further. Ask Claude to find the gaps:

What's missing from this project? Based on what you see in the files, what haven't I thought about? What should exist that doesn't?

Then ask Claude to build what's missing. A timeline, a risk list, a stakeholder summary — whatever the project needs, Claude can create it and save it right to your folder. The file shows up on your computer, in the project folder, ready to open in whatever application you'd normally use.

This is the rhythm you'll settle into: Claude reads your files, you tell it what to build, it creates the output, and the output lives alongside everything else in your project folder. No copying and pasting out of a chat window. No downloading files and moving them manually. It just works.

Part 3: Make It Automatic

Here's where Claude stops being a tool you use and becomes something that works alongside you.

You're going to build a scheduled task — something that runs on its own, on a schedule you set, without you having to open Claude and ask for it.

Think about what would be useful to have waiting for you every morning — a project status update, a summary of what changed yesterday, a list of what's due this week, a reminder of upcoming deadlines.

Here's how to set one up:

I want you to create a scheduled task that runs every morning at [time]. Each day, look at my project files and tell me: what progress was made yesterday, what's planned for today, and what deadlines or milestones are coming up in the next week. Keep it short — I want to read it in two minutes over coffee.

Claude will set up the task. The next morning, it runs. You open Claude and there's a summary waiting for you — not because you asked for it, but because you told it to show up.

Start with one scheduled task. Keep it simple. Once you see how it works, you'll start thinking about what else could run on its own. A weekly project summary for your boss. A daily review of your inbox. A Friday wrap-up that captures what got done and what carried over.

I started with a daily check-in for one project. Within a week, I had one for every active project. Then I replaced the individual check-ins with a single morning briefing that covers everything — progress across all projects, updated timelines, and a prioritized work plan for the day. That evolution happened naturally because once you see Claude working on its own, you start asking "what else could it do while I'm not looking?"

Plugins extend this even further. They give Claude the ability to interact with tools and services beyond your local files. I use plugins to manage my website — Claude handles the HTML updates and pushes changes through a deployment tool without me touching the other tool or even understanding exactly what it is doing or how it works. I built a custom tool that runs daily progress checks across all my projects and builds a dashboard view. The plugin ecosystem is growing, and it's worth exploring what's available for the tools you already use. But the foundation is what you just built — a project, a connected folder, and a scheduled task. Plugins are how you make it do more.

— — —

What Just Happened

You just gave Claude a workspace and a schedule. That's a bigger shift than you think.

Up through Chapter 6, Claude was a conversation partner. You opened it, told it what you needed, got the output, and closed it. The work happened in the chat window. Now Claude has a place on your machine — a folder full of your real files, project-specific rules that shape how it works, and a task that runs whether you're there or not.

Here's what I want you to take away.

The project structure matters. When everything lived in one conversation space, Claude was slower and less focused because it had too much to look at. Giving it a dedicated workspace with only the relevant files and rules made it sharper. It's the same reason you don't spread every document you own across your desk when you sit down to work on one thing.

The scheduled task is the part that surprised me. I set up a daily check-in because I thought it would be convenient. What I didn't expect was how much it would change my morning routine. Instead of spending the first thirty minutes of my day figuring out where I left off and what needed attention, I had a summary waiting for me. Claude had already looked at the progress, done the math on the timeline, and flagged what mattered. I just read it and started working.

In my experience, that's the moment Claude stops feeling like an app and starts feeling like a coworker. Not because it got smarter, but because it started showing up with work done before you sat down at the computer.

Now Go Make Something

You've set up a project and a scheduled task. Now build your system.

Here are ideas. Pick one.

If you're managing multiple projects: Set up a project for each one. Connect the folders. Then build a master morning briefing that covers all of them — progress, timelines, priorities, and what needs your attention today.

If you're a student: Create a project for each class. Drop in your syllabus, notes, and assignments. Set up a weekly task that reviews upcoming deadlines and suggests a study plan based on what's due and how much prep each item needs.

If you run a business: Set up a project for your business operations. Connect your planning documents, financials, and client files. Schedule a Monday morning briefing that reviews the week ahead — deliverables due, meetings coming up, and anything that slipped from last week.

If you're working on a creative project: Create a project for it. Drop in everything — outlines, drafts, research, inspiration. Ask Claude to read through it and build you a progress report. Then schedule a check-in that tracks where you are against where you want to be.

If you just want to get organized: Pick the messiest area of your digital life — the folder with 200 unsorted files, the project that exists in six different documents, the notes scattered across three apps. Create a project, dump it all in, and ask Claude to make sense of it. You'll be amazed at what it finds.

If you're following along with this book: Create a project for the exercises you've been building. Put your Chapter 2 project plan, your Chapter 4 research, your Chapter 5 artifacts, and your Chapter 6 presentation in one folder. Set up a scheduled task that reviews your progress through the book

and suggests what to work on next.

— — —

Quick Reference Card

Claude on your computer — from conversations to projects

What to do	How to do it	Why it matters
Create a project	New project → name it → connect a folder	Gives Claude focused context instead of everything at once
Set project instructions	Write rules in the project settings	Claude follows project-specific guidelines automatically
Work with files	Ask Claude to read, summarize, or create files in your folder	Output goes directly to your computer, not just the chat
Schedule a task	"Create a scheduled task that runs every [time/day]"	Claude works on its own schedule, not just when you ask
Explore plugins	Browse available plugins in settings	Extends what Claude can connect to and automate

Setting up project instructions:

This project is [what it is]. The audience is [who]. Use [tone]. The key files are [list]. The goal is [what we're trying to accomplish]. [Any specific rules — formatting, vocabulary, constraints].

Setting up a scheduled task:

Create a scheduled task that runs every [morning at 7am / Monday at 9am / Friday at 5pm]. Each time it runs, [what you want it to do — review files, summarize progress, check deadlines, build a report]. Keep it [length — one paragraph, one page, a bullet list].

The progression:

Conversations → Projects → Scheduled tasks → Plugins. Each step gives Claude more context and more autonomy. Start with a project. Add a scheduled task when you're ready. Explore plugins when you want Claude to reach be

Chapter 8: Connect your world

The Story

One of the things I had been putting off was building my website.

I'd tried before. The results weren't pretty. I could get something up, but it never looked like something I'd actually want people to see. It looked like what it was — a guy who doesn't build websites trying to build a website.

Then I called my friend Chuck. He'd built a gorgeous site for a mutual friend of ours, and I wanted to know how he did it. He laughed. "I didn't do it," he said. "Claude did."

That stuck with me. My first session with Chuck had been a bit of a blur — he was at the keyboard, telling Claude to connect to various services we needed for the poker app we were working on together. I'd watched but hadn't fully registered what was happening. Claude wasn't just writing code. It was reaching out and connecting to things — source code repositories, deployment services, databases. Tools I'd heard of but never touched.

So I sat down one afternoon without much of a plan. I had just finished the branding for my new company — name, logo, color palette, the whole package. A website was next on the list. I opened Claude and typed something like: build me a web page.

I started by uploading my branding materials and giving Claude a picture of what I wanted. This wasn't going to be a static page I'd update once a year. It had to be functional — a way to collect email addresses, a place to promote and sell my work. I needed a way to automate everything on the website, and that meant finding the right tools.

Claude picked the tools. It looked at what I was trying to build and recommended the services best suited for each piece — Cloudflare to manage DNS, mail records, web hosting, and push updates live from Claude. MailerLite to capture email addresses and manage campaigns. Stripe to handle direct book purchases. And Desktop Commander to wire it all together without me having to touch a terminal or figure out configuration files. Claude didn't just tell me what to sign up for. It connected them. In a couple of hours, I had my first placeholder webpage live on the internet.

It wasn't perfect. The first version never is. But it was real — a live site with my branding, a working email signup, and the foundation for everything else I wanted to add. Like everything with Claude, every revision has made it better.

That experience taught me something I should have learned from watching Chuck. Claude doesn't just work with what you give it. It reaches out and connects to the tools and services you need. And

once those connections are in place, Claude can do things that would have taken me weeks to figure out on my own — if I could have figured them out at all.

This chapter is about those connections. Not building a website — we'll get to that in the next chapter. This is about giving Claude the ability to reach beyond your files and into the tools you already use every day.

Build It: Connect Claude to Your World

In Chapter 7, you gave Claude a workspace — a project folder, project-specific rules, and a scheduled task. That was a big step. But Claude was still limited to what's inside that folder. If you wanted it to know about an email you received or a task on your project board, you had to copy and paste it in.

That's about to change. You're going to connect Claude to tools and services outside your computer, and when you're done, Claude will be able to read your email, check your project boards, and pull from multiple sources to give you a picture of your day — all without you copying a single thing.

Part 1: Desktop Commander — Let Claude Do It

Before we connect to outside services, there's one connection that changed everything for me — and it's right on your own machine.

Here's the problem. Claude is great at telling you what to do. "Create a folder called projects." "Open the terminal and run this command." "Navigate to this settings panel and change the configuration." All very helpful — except when you don't know how to do the thing Claude is telling you to do. I'd find myself staring at instructions thinking, that sounds simple, but where do I even start?

Desktop Commander fixes that. It gives Claude the ability to actually do things on your computer — create folders, move files, run commands, check on running processes — instead of just telling you to do them yourself.

To install it, open your Claude desktop app and go to Settings. Look for the MCP Servers section — MCP stands for Model Context Protocol, but you don't need to remember that. It's just the system Claude uses to connect to tools. Search for Desktop Commander and install it. Claude will walk you through authorizing the permissions it needs.

Once it's installed, try it:

Create a new folder on my desktop called "Claude Test" and put a text file in it with today's date and a note that says "Desktop Commander is working."

If Desktop Commander is set up correctly, that folder and file will appear on your desktop. Not in the chat window. On your actual computer. Claude did it — you didn't have to figure out where the file explorer was or what the right command was or any of it.

Now try something more useful:

Look at my Documents folder. Tell me what's in there, how it's organized, and what looks like it could use some cleanup.

Claude will scan your files and come back with an honest assessment. And if you want it to actually reorganize things — move files into folders, rename things, clean up duplicates — it can do that too. You just have to say yes.

In my experience, Desktop Commander is the difference between Claude being an advisor and Claude being an assistant. An advisor tells you what to do. An assistant does it. Most of the time, I don't need Claude to explain how to create a folder structure. I need it to create the folder structure.

Part 2: Connect Your Email

Now let's go beyond your computer.

The first outside connection worth making is your email. If you're like me, a huge part of your day is driven by what lands in your inbox. Connecting Claude to Gmail means it can read your messages, search your threads, and summarize what matters — without you forwarding anything or copying text into a chat.

Here's how to set it up, step by step.

Open the Claude desktop app and go to Settings. Look for the Connectors section — this is where Claude manages connections to outside services. Click Add Connector and search for Gmail. When you select it, Claude will open a browser window with Google's standard permissions screen — the same kind you've seen if you've ever connected a third-party app to your Google account. Review what Claude is asking for (read access to your messages), approve it, and close the browser window.

Back in Claude, you should see Gmail listed as an active connector. If it doesn't show up right away, close the settings panel and reopen it — sometimes the interface needs a moment to catch up.

Now open a conversation in your project and test it:

What's in my inbox right now? Give me a summary of the most important messages from the last 24 hours.

This is the moment it clicks. Claude isn't guessing what your day looks like based on what you told it. It's looking at your actual inbox and telling you what's there. It can identify which messages need a response, which ones are informational, and which ones you can probably ignore.

Push it further:

Are there any email threads from the last week where someone is waiting on a response from me?

That's the kind of question that would take you ten minutes of scrolling to answer yourself. Claude does it in seconds because it can search across your entire mailbox.

A few days after I connected my email, I looked at my daily summary and noticed a deadline for something I had totally missed. The deadline was that same day, but I had enough time to scramble and get it done. If Claude hadn't put it in my morning briefing, I would have missed it completely. That was the moment I stopped thinking of the email connection as convenient and started thinking of it as essential.

Part 3: Build a Morning Briefing That Actually Knows Your Day

Here's where the connections start working together.

In Chapter 7, you built a scheduled task — a daily check-in that reviews your project files and gives you a summary. That was useful. But it only knew what was in your project folder. It had no idea what was in your email, what meetings were coming up, or what tasks were assigned to you in other tools.

Now that Claude can see your files and your email, you can build something better. But before you write the prompt, think about what you actually need every morning. What's the first thing you check when you sit down? For me, it's three things — where do my projects stand, what came in overnight that needs attention, and what should I work on first. Your list might be different. Maybe you need to know what meetings are on the calendar. Maybe you need a summary of customer messages. Whatever it is, that's what goes in the briefing.

Start with this:

I want to update my morning briefing. Every day at 7 AM, I want you to: 1. Check my project files and give me a progress update — what got done yesterday, what's planned for today. 2. Check my Gmail and summarize anything important that came in overnight — flag anything that needs a response. 3. Give me a prioritized plan for the day based on everything you see. Keep it short. I want to read it in three minutes with my coffee.

When this runs the next morning, you'll open Claude and find something like this waiting for you:

"Good morning. Three project updates: the book manuscript is at 75% with Chapter 8 due today. Two emails need attention — a response to your editor by end of day and a contract question from a client that came in at 11 PM. Based on your deadlines and what's in your inbox, here's your priority order: editor response first, then Chapter 8 drafting, then the client contract after lunch."

That's what a connected briefing looks like. Not just a file summary — a picture of your whole day built from multiple sources.

The first version won't be perfect. Run it for a few days and pay attention to what's missing and what's noise. Maybe it's giving you too much detail on emails that don't matter. Maybe it's not flagging deadlines early enough. After a few mornings, refine the prompt:

Update my morning briefing. I don't need a summary of every email — only flag messages where someone is waiting on me or there's a deadline within 48 hours. Add a "heads up" section for anything coming up in the next three days that I should be thinking about.

This iteration loop is how you make the briefing yours. Everyone's morning is different, and the first prompt is a starting point, not a finished product.

If you want to add more connections, the process is always the same. Tell Claude what you want to connect. It finds the right connector, walks you through authorization, and once it's live, that source becomes part of what Claude can see and work with. Calendar, project management tools, cloud storage — the list of available connectors keeps growing. Each one gives Claude a wider view of your world.

Don't try to connect everything at once. Start with what you connected today — Desktop Commander and your email. Use them for a week. Let the morning briefing run. You'll quickly figure out what's missing from the picture, and that's when you add the next connection. The best system is one you build gradually based on what you actually need, not one you set up all at once hoping you got it right.

Sometimes things don't work the way they're supposed to on the first try. A connection might fail, an authorization might not take, or a service might need configuration you didn't expect. If that happens, Claude may already be using Desktop Commander to find a workaround. If not, start fresh and ask Claude to troubleshoot the problem and find another way. There's almost always another way.

What Just Happened

You just gave Claude eyes and hands beyond your computer.

Think about how far the relationship has come. In Chapters 1 through 6, Claude was a conversation partner — you asked questions, it answered. You pasted in content, it worked with what you gave it. In Chapter 7, you gave Claude a workspace and a schedule, and it started showing up with work done before you sat down. Now you've given it reach. Claude can act on your machine, read your email, and pull from multiple sources to help you plan your day. You're not managing a tool anymore. You're building a system.

The technology behind these connections is called MCP — Model Context Protocol. You don't need to remember that, and you definitely don't need to understand how it works under the hood. What matters is the concept: all of these tools open up a connection to your work and the applications you use. Claude links them together and helps you create a system that makes you more effective and efficient. The pattern is always the same — connect, authorize, use.

Here's what surprised me. I expected the individual connections to be useful. And they are — having Claude read my email saves me time every day. But the real value showed up when I combined them. The morning briefing that pulls from multiple sources is qualitatively different from one that only looks at files. It's the difference between asking a coworker who only sees your project folder and one who also reads your email and checks your calendar. The second one gives better advice because they have more to work with.

In my experience, two or three well-chosen connections are worth more than ten you set up and forget about. Pick the tools where you spend the most time, connect those first, and build from there.

One thing doesn't change as the system gets bigger — you still need to verify the work. The habits from Chapter 3 matter more now, not less. When Claude is pulling from your email and your calendar and your files, a mistake in one source can ripple through everything it builds on top of it. Trust the system, but check the outputs.

Now Go Make Something

You've connected Claude to your computer and your email. Now make those connections work for you.

If you're drowning in email: Build an email triage system. Have Claude scan your inbox every morning and sort messages into three buckets — needs a response today, can wait, and informational only. Then have it draft responses for the urgent ones and put them in front of you for review.

If you manage projects across tools: Connect your project management tool — Asana, Trello, Linear, whatever you use. Then build a weekly status report that pulls from your project board and your email threads about that project. One summary, two sources, zero manual assembly.

If you're a student: Connect your school email. Set up a daily task that checks for assignment updates, professor announcements, and group project messages. Have Claude build a study plan for the week based on what's actually due and what correspondence is happening.

If you run a small business: Connect your business email and set up a customer inquiry tracker. Claude scans for new inquiries, categorizes them, and drafts initial responses. You review and send. What used to take the first hour of your day takes five minutes.

If you're building on what we did in Chapter 7: Take your scheduled task from last chapter and upgrade it. Connect at least one outside source — email, a project tool, or your calendar — and rebuild the morning briefing to include it. Compare the old version to the new one. That gap between them is the value of connected data.

If you want to go further: Think about what other doorways would help. Cloud storage so Claude can find documents in Google Drive or Dropbox. A calendar connector so your morning briefing includes today's meetings. Slack or Teams so Claude can catch up on conversations you missed. Each connection follows the same pattern — tell Claude what you want to connect, authorize it, and it's live.

Quick Reference Card

Connect your world — from local files to connected tools

What to do	How to do it	Why it matters
Install Desktop Commander	Settings → MCP Servers → search and install	Claude can act on your computer, not just advise
Connect Gmail	Settings → Connectors → Add → Gmail → authorize via Google	Claude reads your email without copy-paste
Build a morning briefing	"Create a scheduled task that checks my files and email every morning"	One summary, multiple sources, zero manual work
Add more connectors	Tell Claude what tool you want to connect → authorize	Each connection gives Claude a wider view
Manage connections	Settings → Connectors → review active connections	Keep track of what Claude can access

The connection pattern (it's always the same):

Tell Claude what you want to connect → Claude finds the right connector → you authorize access → it's live. Works for email, calendars, project tools, cloud storage, and more.

Building better briefings:

Update my morning briefing to include [new source]. Every day at [time], check my [files / email / calendar / project board] and give me: what needs attention today, what's c

Chapter 9: Build something that lives on its own

The Story

A live website and a real website are not the same thing.

By the end of that first afternoon, I had a page on the internet. My logo was on it. My colors were on it. My picture was there, with a description of what we were building, a cover image, and a blurb about each of the books I was working on. Technically, it was live — anyone in the world could type the address and land on it. And if they had, they would have seen exactly what it was. A placeholder. One screen, a headline, my picture and bio and a plug for my books. It didn't really say or do much.

It was a start. The old version of me would have stopped there and called it good enough for now, and I would have pushed it off until I needed it yesterday. That wasn't enough this time. I had just started a new company, Marque Publishing, with a real name and real branding and real books I wanted people to buy. The placeholder didn't say any of that. It said "I stood up a web page and it isn't very good."

So, I went back to Claude and described the actual thing I wanted. A professional-looking website that represented my brand. It needed a books page with the ones I have in the works, an about page, a blog, so I'd have somewhere to think out loud. And a way for the right reader to leave their email and hear from me. I didn't draw it. I didn't sketch a layout or pick fonts or worry about how any of it would get built. I just described it the way I'm describing it now, and I asked Claude to build it.

It did. Not all at once, and not perfectly — the first version of anything never is. I'd look at a page, tell Claude what felt off, and watch it change. This wording is clunky. Move that up. The spacing is too tight. Each pass took a minute or two, and each pass made it better. One screen grew into a real site that captured the image I wanted to project, with a page for everything I have to offer and a way to interact with visitors.

Then came the part that actually mattered. I told Claude to push it live, and it did — the same way it had set up the placeholder, working through the tools I'd already connected. Cloudflare for hosting. The signup wired to MailerLite, handing a free guide to anyone who joined. The whole thing out on the internet under my own domain.

Here's the moment I remember. I had just wired the signup to MailerLite, and I tested it the way you test anything — I typed in an email address and hit subscribe. A few seconds later, a welcome email landed in the inbox. I hadn't written it that morning or pressed send. The site had done it. And it would do the same for the next person, and the next, handing each one the free guide and starting the

conversation — whether I was at my desk or sound asleep.

That's when it clicked. I hadn't just made a document or a chart or a summary — something I open, use, and close. I had made a thing that lives on its own. It runs without me. It does its job whether I'm watching or not.

And the site keeps getting better, because it keeps getting fed. The writing I do with Claude doesn't just sit in a folder — when something's worth sharing, I push it from Claude straight to the blog, and the site has fresh content without me opening an editor. MailerLite quietly gathers the email addresses of people who want to hear when there's more. A website like this is never really finished, and that's the point: it grows a little every week, and it works for me in between.

My son Jayden — the one who was anti-AI from the start — is listed on the website as head of game development and engineering. When I showed him the new website, the first thing he said was none of the headings were capitalized properly. Every word should have its first letter capitalized, he said. I saw my opening. I pulled up the website project in Cowork and said, you tell it what the proper capitalization rules are. He typed the prompt. I suggested one small change. He hit enter, and about three minutes later it was done. He said "that's pretty cool" and got up and walked out of my office.

I thought back to that conversation with Chuck about building a site for a friend, when he told me Claude had done it, not him. At the time it sounded like a magic trick I'd never pull off myself. Now I had pulled it off myself.

This chapter is how you do it too.

Build It: Make Something That Runs Without You

Everything you've built so far in this book, you open and use. A document. A chart. A briefing waiting for you in the morning. Useful, all of it — but it sits still until you pick it up.

Now you're going to build something different. A thing that exists out in the world and does its job on its own. We'll build a website, because it's the clearest example and the one most people actually want. But the same moves work for any small tool you can imagine — a calculator, a sign-up page, an interactive guide. The pattern is what matters, and the pattern is the same every time.

Here it is, start to finish: describe what you want, let Claude build it, look at it and react, get it live, and put it in front of a real person. Five moves. Let's walk them.

Step 1: Describe the Thing, Not the Code

Open a conversation in your project and tell Claude what you want to build. Not how to build it — what it should be and who it's for. The more Claude understands the purpose, the better the first version will be.

Don't reach for technical words. You don't have any, and you don't need any. Describe it the way you'd describe it to a friend who happened to know how to build websites.

Here's the kind of thing I mean:

I want to build a simple personal website. It's a home base on the internet for me — somewhere I can point people when they ask what I do. One page is fine to start. I want a short intro at the top that says who I am, a section about the work I do, and a way for people to get in touch. Keep it clean and modern, not busy. Build me a first version and show me what it looks like.

Notice what's in there and what isn't. There's a purpose — a home base I can point people to. There's an audience — people who ask what I do. There's content — intro, work, contact. And there's a feel — clean, not busy. That's everything Claude needs. There's not one word about how any of it gets made, because that's Claude's job, not yours.

If you have branding — a logo, colors, a headshot — say so and upload it, the way I uploaded my own materials. If you don't, don't worry about it. Claude will make reasonable choices, and you can change them later. It will even help you create the brand if you delegate the task to it.

Step 2: Let Claude Build It

Now Claude goes to work, and you watch.

In a few moments you'll have a working version you can actually look at — not a description of a website, an actual website. Claude builds the whole thing, every piece of it, and shows it to you. This is the part that used to take a person with technical skills days or weeks. It now takes about the length of time it takes to read this paragraph.

Look at it. Really look. It won't be right yet, and that's not a problem — that's the process. The first version is Claude's best guess from your description. Your job now is to react to something real instead of trying to imagine it from scratch, which is a much easier job.

Step 3: React in Plain Language

This is where the thing becomes yours, and it's the step people underestimate.

You don't fix a website by knowing how to fix websites. You fix it by telling Claude what's wrong in plain words, the same way you'd tell a designer sitting next to you. You don't have to be precise or technical. "I don't like that" is a complete and useful instruction.

Things you might say:

Make the headline bigger and move it higher up.

The colors feel cold. Warm them up — something friendlier.

The contact section is too far down. People won't scroll that far. Move it up.

Add a photo at the top, and make the whole thing feel less corporate.

Each time, Claude makes the change and shows you the result. You react again. This back-and-forth is the entire skill, and you already have it — you've been doing it since Chapter 2. Building a website turns out to be the same conversation you've been having all along, pointed at a different kind of thing.

Keep going until you look at it and think, yes, that's it. Don't aim for perfect on this pass. Aim for good enough to show somebody. You'll learn more from one real person looking at it than from another hour of fiddling on your own.

Step 4: Get It Live

A website on your screen is a draft. A website anyone can visit is the real thing. This is the step that makes it live on its own, and it's simpler than you'd expect.

Tell Claude you're ready:

This is ready to go live. I want it on the internet so I can send people the link. Walk me through what I need and set it up.

Getting a site onto the internet takes two things: a place to host it and an address people can type. Claude will recommend both, and if you connected your tools the way we did in Chapter 8, it can do most of the setup for you rather than just telling you how. When I did this, Claude used Cloudflare to host the site and manage the address, and it handled the configuration I would never have figured out on my own. You'll have to make a couple of decisions — what web address you want, mostly — and approve a few steps. Claude handles the rest.

A word of honesty here, because this is the one step with real-world friction. A custom web address with your own URL requires registration and an annual fee. Setting it up involves a few minutes of clicking through a service. That's normal. It's the cost of owning a real place on the internet instead of borrowing one.

I suggest going to Cloudflare, at domains.cloudflare.com. There are plenty of other places you can register, but Cloudflare seemed to be less expensive, and they offer free website hosting. Come with more than one idea for a name, because as you search for available names, you will find many of the names you like are already taken.

If you're not ready for that, you don't have to stop. Tell Claude you want to share it without buying a domain yet, and ask for the simplest way to get a link you can send to a few people. There's almost always a free path to get something in front of others before you commit. When Chuck and I built the poker app, we used a free hosting service from Vercel. The goal right now is to get it out of your screen and into someone else's hands. You can buy your own domain once you're sure you love it.

Step 5: Put It in Front of a Real Person

Now do the thing that turns a project into something real. Send the link to one person and watch what happens.

Not ten people. One. Someone whose honesty you trust. Send them the link and ask them to do the thing the site is for — read it, find your contact info, sign up, whatever it is. Then pay attention. Where do they get confused? What do they miss? What do they ask you that the site should have answered?

This is what the framework means by testing with real people, and it's worth more than any amount of solo polishing. You are too close to your own creation to see it clearly. The first time my wife looked at my site, she asked where the books were — and they were right there, but not where she expected them. That one comment fixed a problem I'd have never seen myself.

Take what you learn back to Claude and fix it:

*A friend looked at this and couldn't find the contact section, and said the top felt too crowded.
Fix both.*

Then send it to one more person. This loop — show it, learn, fix, show it again — is how everything good gets made. The version that started as a placeholder on my screen became a real website the same way: not in one heroic afternoon, but through a dozen small rounds of someone looking and me fixing.

That's the whole build. Describe it, let Claude make it, react in plain words, get it live, and let real people show you what's left to fix. Five moves, and a thing now exists in the world that wasn't there before — running on its own, doing its job while you do something else.

The Same Five Moves, a Different Thing

A website is the obvious thing to build, but it isn't the only thing. The same five moves work on anything you can describe.

Here's the one that grabbed me. When I started Marque, I assumed I'd only ever publish my own work. But the more I build, the more I think about the other authors out there who assume building anything real takes a skill they don't have — the way I used to. So I made them something: a calculator where a writer enters their page count, the number of copies they want, and a cover price, and it shows what printing will cost and what they'd earn per book. Numbers in, an answer out. That's a different animal from a web page — a small machine with logic inside it, the kind of thing I used to assume was real programming.

It was the same five moves. I described it, Claude built it, and I went a few rounds telling it what was off until the numbers came out right. The prototype I ended up with is far from perfect, but it works, and it didn't exist before I asked for it.

What I think makes it worth the extra effort is that this is the kind of tool I would have liked to have when I started figuring out what it would take to self-publish a book. It confirmed one thing quickly. I am not going to give up my day job anytime soon.

That's the thing to hold onto: a tool with real math inside gets built exactly the way a page does. Describe, build, react, ship — pointed at a calculator instead of a page.

It doesn't have to be a calculator at all. It could be a quiz that sorts people into the right product, a sign-up page for an event, a scoreboard for your kid's team, an interactive guide for your customers. The tool is different every time, but the five moves are exactly the same ones you just used.

— — —

What Just Happened

You just built and shipped real software, and you never saw a line of code.

Stop and sit with that, because it's bigger than a website. For most of computing history, the line between people who use software and people who make it was a wall. On one side, everybody. On the other, programmers — people who'd spent years learning languages most of us can't read. If you had an idea for a tool or a site or an app, you either learned to code, hired someone who could, or let the idea die. Most ideas died.

That wall is gone. You just walked through where it used to be. You described what you wanted in plain English, and the thing got built. Everything that used to require a programmer — the building, the wiring, the getting-it-online — happened on the other side of a conversation you were fully able to have.

There's a name for what you just did. People call it vibe coding. You describe the application or tool you want in plain words. You set the parameters and what you want it to do. The more context you give it, the better the first iteration will be. Claude writes the actual code, and you steer by reacting to what comes back, without ever reading the code yourself. It started as a half-joke among programmers. It turned into the most natural way for the rest of us to build.

In my experience, this is the part people struggle to believe until they've done it themselves. We've been told for so long that making real things with computers is hard and technical and not for us. So, when it turns out to be a conversation, the instinct is to assume it can't have been real, or that what you made doesn't count. It's real, and it counts. A stranger signing up on my site didn't know or care that I'm not a programmer. The site worked. That's all that matters to the person on the other end.

The deeper shift is the one in the chapter title. Up to now, everything you made with Claude was something you operate — you open it, it helps, you close it. What you built here is different in kind. It runs without you. It greets visitors, collects emails, sells your work at two in the morning while you sleep. You didn't make a tool you use. You made a tool that works on your behalf.

And the way you made it is the same way you've made everything else in this book. Describe, look, react, repeat. Building a website turned out to be the skill you already had, aimed at something you never thought you were allowed to point it at.

— — —

A Word About Claude Code

Some of you have heard about Claude Code and are wondering why we never touched it. Fair question. It comes up every time I tell another author or small-business owner what I've been building, usually phrased as, "Don't you need to be a programmer for that?"

Here's the honest answer. Everything in this book — the website, the tools, the briefings, all of it — I built in Cowork, just by describing what I wanted. Not once did I open a programmer's tool or type a command into a black screen. I want you to consider that, because the belief that real building requires programming skills is exactly the wall this chapter has been about.

It helps to know how the pieces fit. Everyone starts in Chat — in the app or on the web. It's where you ask quick questions, kick ideas around, and run Deep Research, the way we did back in Chapter 4. When a chat turns into something you want to keep working on in its own space, you create a new project in Cowork, where it gets its own files and tools. Have Claude package up your chat history and any associated files so you can move them into the project. That's where everything in this book got built. And underneath Cowork is Code — Claude Code itself, with its full power in your hands.

Here's the part people get backwards. Code isn't a walled garden for professional programmers. It's the opposite: it hands you the same tools the professionals use, without asking you to become one. It's where you go when a project needs to grow up into real software — more moving parts, more polish, something built to last. You don't need a degree to walk through that door. You need what got you this far: the willingness to describe what you want and react to what comes back.

So, when would you reach for it? For most of what we make, Cowork is fine, and that's where the work gets done. You'll know the day you outgrow it: a project gets big enough that it strains at the edges of a conversation, and you find yourself stepping in on things Claude should be handling on its own. That's the signal to move up to Code — and the reassuring part is that you're already using it. Cowork is the friendly wrapper around Code, with more power waiting underneath. The tools change, but the way you work with them stays the same: describe, look, react, repeat. Even in Code, you're still vibe coding — describing what you want, steering by what comes back — and the tool just builds something sturdier with it.

That's how I know my own day is coming. The calculator I prototyped earlier is real. Cowork got it working and convinced me there's a place for it. But building it out properly with the features I want to put in front of other authors is a job for Code. So, that's my plan: my first time there, after a whole book of working in Cowork.

Until your day comes, this is your workshop — and you've already proved you know how to use it.

— — —

Now Go Make Something

You can build things that run on their own now. The question isn't whether you can — it's what's been sitting on your someday list waiting for permission. Here's where to start.

If you've been meaning to build a personal site: Do it this week. A home base on the internet — who you are, what you do, how to reach you. The thing you've been telling yourself you'll get to. You'll have a first version live in an afternoon.

If you run a small business: Build a landing page for one product or service. Describe what you sell, who it's for, and what you want visitors to do — call, book, buy, sign up. Get it live and send it to five customers before you spend a dollar on advertising.

If you're a teacher: Build an interactive page for your class. A study guide students can click through, a quiz that gives them answers as they go, a simple page where parents can find this week's assignments. Something that works for your students even when you're not in the room.

If you're an entrepreneur testing an idea: Build a one-page site that describes the thing you're thinking about making and asks people to sign up if they're interested. Put it in front of real people and watch whether anyone bites. You'll learn more from twenty signups, or zero, than from a month of guessing.

If you have kids: Build them something. A simple game with their name in it, a chore chart they actually want to use, a page about the thing they're obsessed with this month. Build it together and let them tell you what to change. Watching a kid realize they can make the computer do what they imagine is worth the afternoon by itself.

If you organize anything — a club, a team, a group: Build a page that holds the things people always ask about. The schedule, the rules, the sign-up form, the contact list. One link you can send instead of answering the same question forty times.

If you want to go further: Pick the most useful thing you made and keep building on it. Add a page. Add a feature. Connect it to one of the tools from Chapter 8 so it does more than sit there. The first thing you ship is a starting line, not a finish line — and now that you know the moves, the second one is faster.

Whatever you pick, get it in front of one real person before you call it done. That's the step that turns something you made into something that works.

— — —

Quick Reference Card

Build something that lives on its own — in Cowork, no code

The move	What you do	Why it matters
Describe it	Tell Claude what to build and who it's for — in plain words	Purpose and audience drive a better first version than any layout sketch

Let it build	Watch Claude create a real, working version you can look at	The hard part — the actual building — is Claude's job, not yours
React	Tell Claude what's wrong in plain language: "bigger," "warmer," "move that up"	This is the whole skill, and you already have it
Get it live	"Set this up so I can send people the link" — Claude handles hosting and setup	A site on your screen is a draft; a site anyone can visit is real
Test it	Send it to one trusted person and watch what confuses them	You're too close to see it clearly; one real person shows you what's left

The build, in one breath:

Describe what you want → let Claude build it → react in plain words → get it live → put it in front of a real person → fix what they trip on → repeat.

Getting it live, honestly:

A custom web address costs a little money each year and takes a few minutes to set up. Worth it when you're sure. Not ready? Ask Claude for the simplest free way to share a link first, and buy the address once you love the thing.

Chat, Cowork, and Code:

Claude gives you three ways to work, each opening up more power than the

Chapter 10: Make it yours forever

The Story

Eight days ago, I had an idea. My uncle was asking me about how I use AI and if I had any suggestions for helping him do more with it in his business. Chuck and I have talked for months about onboarding friends into Claude. That was when I started mapping out this book. It started with a couple of pages of notes. In hindsight, I should have just started with a chat in Claude about the need for this book.

My goal was to create something that connects the dummy's level books with DIY YouTube videos without the video. I researched what was already out there and found a lot of books about how Claude works, but nothing that would help someone go from their first session at the keyboard, to creating things that matter in ten sessions. I gave Claude my notes and described the vision of what I wanted to create. I started with a framework document, a set of drafting rules, and a few good chapter ideas.

Five days ago, I was satisfied that this was a viable project that the market needed. So, over my morning coffee at 6am, I created the project folder and said let's go. Here we are approaching midnight on the fifth day and I have a finished manuscript.

I need to explain how that happened, because it didn't happen the way you might think.

Claude didn't write this book. I've said that before and I'll keep saying it, because it matters. What Claude did is what it's been doing for you across every chapter — the heavy lifting. The structure, the formatting, the iteration at a pace I couldn't match on my own. But every story in this book is mine. Every opinion is mine. Every awkward sentence I insisted on keeping is mine. Claude is the vehicle. I'm the driver.

What made five days possible wasn't Claude getting faster or smarter. It was everything I'd built around it.

Here's what my mornings looked like this week. I'd sit down with coffee and open the project. The daily check-in — the one I showed you how to build in Chapter 7 — was already waiting. It knew which chapters were done, which one was next, and where I stood against the timeline. I didn't have to reconstruct any of that. The system remembered.

The project folder had everything Claude needed — the framework, the voice rules, the progress tracker, every finished chapter for context. I didn't re-explain the book every session. I didn't re-establish the rules. I'd built all of that once, and it held.

Before drafting, I'd run what I call a preflight — a quick walk through the scaffolding. What chapter are we writing? Where does it sit in the book? What should the reader walk away with? What's the story? That routine took five minutes and saved an hour of drift. After drafting, I'd run a postflight — a line-by-line check against my voice rules, measuring the output against the quality bar I'd set. Same questions every time. Did it land? Are we at the bar? What did we get right? What could be better?

None of that is complicated. A folder. A checklist. A daily task. A set of rules I wrote down once. But together, they changed what was possible. I wasn't managing Claude anymore. I was creating, and the system was handling everything else.

I thought about that moment from the Introduction — fifteen years of book ideas sitting in random notes and emails to myself, going nowhere. Same guy. Same ambitions. The difference is I built a system, and the system made the ambition executable.

That's what this chapter is about. You've spent nine chapters learning what Claude can do. Now you're going to build something that makes all of it work together — a personal system that's yours, that reflects how you think and what you need, and that keeps getting better the more you use it.

Build It: Your Personal Claude System

You're going to build three things in this chapter. A prompt library — the instructions you've refined that get Claude to do exactly what you need. A set of templates for the work you do over and over. And a workflow that ties it all together so your system runs without you having to rebuild it every time you sit down.

By the end, Claude won't just be a tool you're good at using. It'll be a system you've built for the way you work.

Part 1: Build Your Prompt Library

Think about the prompts that have worked for you. Not the first thing you typed into Claude back in Chapter 1 — the ones you've refined. The ones where you figured out exactly what to say to get the output you actually wanted.

Maybe it's how you ask Claude to write emails in your voice. Maybe it's the way you frame a research question that gets you actionable results instead of a Wikipedia summary. Maybe it's the instructions you give when you want a presentation that doesn't sound like it was written by a committee.

Those refined prompts are valuable. They represent trial and error you've already done. The problem is, most people don't save them. They retype something close to what worked last time, lose the nuance, and wonder why the output isn't as good.

Open your project folder and create a new subfolder called prompts. Then open Claude and start a conversation:

I want to build a prompt library — a collection of my best prompts organized by what I use them for. Help me think through the categories. Here's what I use Claude for most: [list your top 5–10 use cases]. For each one, I want to capture the prompt that works best, why it works, and any specific instructions that make a difference.

Claude will help you organize this. What you're building is a reference you can pull from every time you start a new conversation. Instead of writing a prompt from scratch, you open your library, grab the one that fits, and customize it for the situation.

Here's what made the biggest difference for me: I didn't just save the prompts. I saved the rules. My drafting rules for this book are twenty-two specific instructions about voice, structure, vocabulary, and quality. They tell Claude what my writing sounds like, what AI-default writing sounds like, and how to tell the difference. Those rules are the reason every chapter in this book sounds like me and not like a machine.

Your rules won't look like mine. If you're using Claude for sales emails, your rules might be about tone, length, and what never to say to a prospect. If you're using it for project management, your rules might be about how to structure updates and what level of detail your stakeholders actually want. The format doesn't matter. What matters is that you've written down what works so Claude doesn't have to guess.

Save each prompt as a plain text file in your prompts folder. Name them so you can find them — email-client-update.txt, research-market-analysis.txt, presentation-executive-pitch.txt. When you start a new conversation for that type of work, paste the prompt in and adapt it. Over time, your library grows, and your starting point gets better every time.

Part 2: Create Templates for Recurring Work

Some of the work you do is unique every time. But a surprising amount of it follows the same structure — you just fill in different details.

Weekly status reports. Client proposals. Meeting summaries. Project kickoff documents. Performance reviews. If you're doing the same kind of work more than twice, you need a template.

A template is different from a prompt. A prompt tells Claude what to do. A template gives Claude the structure to do it in. The prompt says "write me a client proposal." The template says "here's what our proposals look like — this section first, then this, then this, in this tone, at this length, with these elements always included."

Start with the piece of work you do most often. Open Claude:

I write [type of document] about [frequency]. Here's an example of a good one I've done before: [paste it]. Help me turn this into a reusable template that captures the structure, tone, and key elements so I can use it as a starting point every time.

What Claude gives you back is a skeleton — the bones of the document with placeholders where the specifics go. Save it in a templates folder inside your project. Next time you need to write one, you

hand Claude the template and the details, and it fills in the blanks in the format you've already validated.

I do this for everything now. This book has a template — every chapter follows the same rhythm. Story, Build It, What Just Happened, Now Go Make Something, Quick Reference Card. I didn't invent that structure for each chapter. I built it once in the framework document, and every chapter follows the pattern. The consistency is the point. It freed me up to focus on what's different about each chapter instead of reinventing the structure every time.

The same principle works for anything you produce regularly. A weekly team email. A monthly report. A quarterly business review. Build the template once. Use it forever. Update it when something changes. That's how you stop doing the same work twice.

Part 3: Set Up Your Workflows

You've already built one workflow — the scheduled daily check-in from Chapter 7. That's a single automated task. Now you're going to connect the pieces into something bigger.

A workflow is a sequence of steps you do regularly, turned into a system Claude can support or run on its own. My writing workflow for this book looked like this: preflight walk, draft the chapter, postflight review, update the progress tracker, archive the previous version, save the new one. Every session, same steps. The more I formalized it, the less time I spent thinking about the process and the more time I spent on the writing.

Think about a process you go through regularly. It might be:

Preparing for meetings — pulling context from your calendar, drafting an agenda, compiling notes from last time, writing follow-up items after.

Processing your inbox — triaging what matters, drafting responses, flagging things that need action, summarizing what you'll deal with later.

Running a weekly review — checking what got done, what slipped, what's coming, and adjusting the plan.

Pick one and map it out. Open Claude:

I want to build a workflow for [your process]. Here's what I do now, step by step: [describe it]. Help me figure out which parts Claude can handle, which parts can be automated with scheduled tasks, and which parts I need to do myself. Then help me set it up.

Claude will walk you through it. Some steps will become scheduled tasks. Some will become prompts you run manually but with a template ready. Some will stay entirely in your hands — the judgment calls, the decisions, the things that need a human. That's fine. The goal isn't to automate everything. The goal is to stop spending time on the parts that don't need you.

Once your workflow is set up, try it for a week. You'll find pieces that need adjusting. Tweak them. The first version is never the last. The system gets better because you keep using it and keep refining

it.

That workflow mindset led me somewhere I didn't expect. I was doing research on ISBNs and barcodes for this book — a straightforward question about whether to buy barcodes from Bowker or just the ISBNs. Claude answered the question, then raised a list of things I'd almost completely overlooked: ISBN assignment and logging, barcode generation, the copyright page, copyright registration, LCCN and PCN applications, BISAC codes, the metadata record, and a checklist for entering all of it at KDP. That was a lot of production work I nearly missed, and a lot to keep straight.

I started thinking — this isn't just for this book. This is the production process for every book I'll ever publish. I didn't want a checklist to follow. I wanted Claude to handle it. So I asked: could you build a skill to do all of this every time I move a final manuscript into production? Claude said it was a perfect thing to build a skill for. Five minutes later, I had one. It told me everything the skill handles and what it doesn't. That conversation led to a second skill — a companion for layout, formatting, and placement of the copyright page, LCCN, dedication, author's note, and foreword. Two skills, built in an afternoon, that took the entire production process off my plate and put it on Claude.

That's what a system looks like when it's really working. You're not following instructions anymore. You're building the instructions once and letting Claude run them.

Part 4: Know Your Models

Claude isn't one thing. There are different models behind it, and which one you use matters more than most people realize.

As of this writing, there are three: Haiku, Sonnet, and Opus. Think of them as different gears. Haiku is fast and lightweight — good for quick answers, simple formatting, routine tasks where speed matters more than depth. Sonnet is the middle ground — capable, reasonably fast, good for most everyday work. Opus is the most powerful — deeper reasoning, better at nuance, better at holding complex context, but slower and it costs more of your usage allocation.

For this book, I used Opus for everything involving voice, structure, and editorial judgment. Editing a chapter, reviewing for quality, making sure the writing kept my voice and didn't sound like AI — that's Opus work. If I were just reformatting a table or generating a quick summary, Sonnet would be fine.

The practical advice is simple. Start with whatever Claude gives you by default. If the output feels shallow, if it's missing nuance you expected, or if it's not holding the complexity of what you asked — switch to a more powerful model. You'll develop a sense for this over time. I didn't know the difference mattered until I saw it side by side.

If you're on the free plan, you may not have a choice. That's fine — do the work with what you have. The techniques in this book work regardless of which model is running. But if you're on a paid plan, pay attention to model selection. For your most important work, use the best tool available.

— — —

What Just Happened

You just built a system. Not a single tool, not a clever prompt — a system that captures how you work and makes Claude a permanent part of it.

Your prompt library means you never start from zero. Your templates mean recurring work follows a proven structure. Your workflows mean the routine parts happen on their own, freeing you to focus on the parts that need your judgment, your creativity, your voice.

That's the shift. In Chapter 1, you had a conversation. In Chapter 9, you built something that lives on its own. Now you've built something that makes all of it work together. The individual capabilities didn't change. How they connect did.

Before you go build everything you're about to build, there's one more thing we need to talk about. And it's not a feature.

The Creator's Responsibility

You're a creator now. That comes with a responsibility that I take seriously and you should too.

When you use Claude to write something, the words go out with your name on them. That means you own what they say. If Claude gets a fact wrong and you don't catch it, that's on you — we covered that in Chapter 3. If Claude helps you write a proposal and it overpromises, your name is on the promise. The tool doesn't absorb the accountability. You do.

Be honest about how you work. I tell people Claude helps me write. I don't pretend I type every word myself. I also don't pretend Claude does the thinking for me. The ideas, the experience, the judgment — that's mine. The execution speed and the ability to produce at a level I couldn't before — that's the partnership. Most people respect that distinction when you're straightforward about it.

Think about what you create and who it affects. Claude will help you build almost anything you ask it to. That's powerful, and power without judgment is how things go sideways. If you're building something that touches other people — a business, a communication, a tool — make sure it reflects who you actually are. The ease of creation isn't a reason to lower your standards. It's a reason to raise them, because now you can.

Use Claude to do more than you thought possible. Just make sure the "more" is something you're proud of.

Now Go Make Something

I started this book as a skeptic who treated AI like a search engine. My son was absolutely anti-AI. I talked to him about the cool things I was doing with it and encouraged him to try it out. The answer was always the same — no thank you. Then he sat down with Claude, created a prompt, and

watched it work. A friend named Chuck showed me what was possible over a Saturday morning, and something clicked.

Since then, I've finished two non-fiction b

Appendix: Your Quick Reference Guide

You finished the book. Now you're in the middle of a project, you need to remember how to do something, and you don't want to reread a chapter to find it. This is the page you flip to.

Everything below is pulled from the ten chapters you just worked through --- the most useful prompts, the capability map, the model guide, and the mistakes worth avoiding. Keep it close.

\-\--

What Claude Can Do --- The Capability Map

Each chapter taught you a capability. Here's what you can now do, where to do it, and which chapter to revisit if you need a refresher.

	Ch
Capability	What you can build
1	Conversations Emails, plans, brainstorm, organized thinking Chat, Cowork
2	Full projects Documents, proposals, plans from blank page to done Chat, Cowork
3	Verification Fact-checked outputs, personal verification checklist Any mode
4	Deep research Cited reports, market analysis, competitive intelligence Chat (Research toggle)
5	Visual creation Charts, flowcharts, dashboards, interactive tools, spreadsheets Chat (artifacts), Cowork (files)
6	Presentations Slide decks, reports, proposals, reframed for different audiences Chat, Cowork
7	Desktop projects Project workspaces, file management, scheduled tasks, plugins Cowork
8	Connected tools Email integration, morning briefings, multi-source workflows Cowork (with connectors)
9	Live things Websites, landing pages, tools that run without you Cowork
10	Your system Prompt library, templates, workflows, model selection All modes

\-\--

Master Prompt Templates

The prompts that work best across the book. Copy, adapt, and add to your prompt library.

The Starter (Chapter 1)

I need to \[what you're trying to accomplish\]. The context is \[relevant background\]. The audience is \[who will see this\]. I want it to be \[tone/style\]. Can you help?

The Brain Dump (Chapter 2)

Here's everything in my head about \[project\]. \[Dump it all --- goals, situation, constraints, timeline, budget, obstacles.\]

The Stress Test (Chapter 2)

Look at this plan as a skeptic. What's most likely to fail? Where am I being unrealistic?

The Anti-Hallucination Rule (Chapter 3)

Do not fabricate information. Do not invent statistics, names, dates, or citations. If you don't have reliable information, say so.

The Research Mission (Chapter 4)

I want you to research \[specific question\]. Here's my situation: \[your context --- role, industry, constraints\]. I plan to use this to \[what you'll do with it\]. Give me \[format you want --- recommendations, comparison, action plan\].

The Visual Request (Chapter 5)

Here's my data: \[paste it\]. Create a \[chart type\] showing \[what you want to see\]. The audience is \[who\]. Make it \[clean/detailed/colorful\].

The Presentation Brief (Chapter 6)

Build a \[number\]-slide deck about \[topic\] with speaker notes. My audience is \[who --- what they care about, what makes them skeptical\]. I have \[time limit\] to present.

The Reframe (Chapter 6)

Rebuild this for \[different audience\]. They care about \[different priorities\]. Don't just adjust the tone --- restructure the argument so it leads with what matters to them.

The Project Setup (Chapter 7)

This project is \[what it is\]. The audience is \[who\]. Use \[tone\]. The key files are \[list\]. The goal is \[what we're trying to accomplish\].

The Scheduled Task (Chapter 7)

Create a scheduled task that runs every \[morning at 7am / Monday at 9am\]. Each time it runs, \[what you want it to do\]. Keep it \[length\].

The Briefing Builder (Chapter 8)

Every day at \[time\], check my \[files / email / calendar\] and give me: what needs attention today, what's coming up this week, and what I might be forgetting. Keep it to \[length\].

The Build Request (Chapter 9)

I want to build \[what it is\]. It's for \[who\]. It needs to \[what it does\]. Keep it \[clean/simple/professional\]. Build me a first version and show me what it looks like.

The Workflow Builder (Chapter 10)

I want to build a workflow for \[your process\]. Here's what I do now, step by step: \[describe it\]. Help me figure out which parts Claude can handle, which parts can be automated, and which parts I need to do myself.

\-\--

Choosing the Right Model

Claude has three models. Each one trades speed for depth. Picking the right one for the task makes a real difference in output quality.

	Model	Best for
Speed	Haiku	Quick answers, simple formatting, routine tasks
Fastest	Sonnet	Speed matters more than depth
Everyday work	Sonnet	Solid reasoning, good balance
Fast Default	Opus	For most tasks
Complex reasoning, nuance, voice, editorial judgment	Opus	
Slower	Opus	Your most important work

The rule of thumb: Start with the default. If the output feels shallow or misses nuance, switch to a more powerful model. For anything where voice, judgment, or complexity matters, go straight to Opus.

\-\--

The Mistakes Worth Avoiding

Every chapter has a version of these. Here are the ones that cost people the most time.

	The mistake	Why it happens	What to do instead
Expecting perfection on the first try	People think AI should nail it in one shot	Treat the first response as a starting point. Iterate.	
Not giving enough context	You know your situation; Claude doesn't	Brief Claude like a colleague on their first day	
Trusting everything without checking	Claude sounds confident even when it's wrong	Verify anything you're going to rely on	
Retyping prompts instead of saving them	It feels faster in the moment	Build a prompt library. Your future self will thank you.	
Using one conversation for everything	It's easier than starting fresh	Separate projects. Focused context gets better results.	
Stopping at "good enough"	The first useful version feels like the finish line	Push two more rounds. The gap between good and great is one more iteration.	
Not saying how you'll use the output	Claude doesn't know if it's for your boss or your notes	Tell Claude the audience and the stakes	
Asking Claude to explain instead of build	Old search-engine habits	Ask Claude to make the thing, not describe it	

\-\--

The Progression --- How It All Connects

Start Creating (Chapters 1--3): Conversations → Projects → Verification. You learn to talk to Claude, build something real, and check the work.

Create More (Chapters 4--6): Research → Visuals → Presentations. You go deeper, build things people can see, and create deliverables you'd put your name on.

Create Bigger (Chapters 7--9): Desktop → Connected tools → Live things. Claude moves from a chat window to a workspace, connects to your world, and builds things that run without you.

Create Forever (Chapter 10): Your system. Prompts, templates, workflows, model selection. Everything you've learned becomes a permanent part of how you work.

\-\--

Where to Go from Here

Claude: claude.ai --- sign up, use the web version, or download the desktop app

Anthropic's documentation: docs.anthropic.com --- detailed guides on everything Claude can do

Anthropic's learning resources: anthropic.com/learn --- courses and tutorials for getting more out of Claude

Prompt engineering guide: docs.anthropic.com/en/docs/build-with-claude/prompt-engineering --- the official guide to writing better prompts

This book's resources: marquepublishing.com --- updates, bonus materials, and the community

The best resource is the one you build yourself. Start with the prompt library from Chapter 10. Add to it every time something works. Share what you learn. And when you have your moment and realize you are creating too, reach out. I want to hear your story.